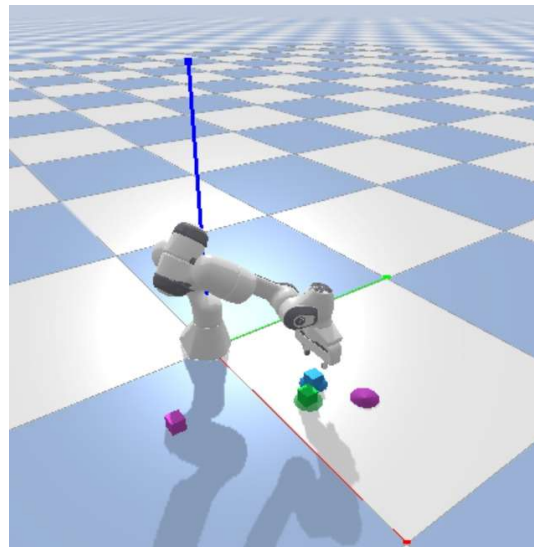
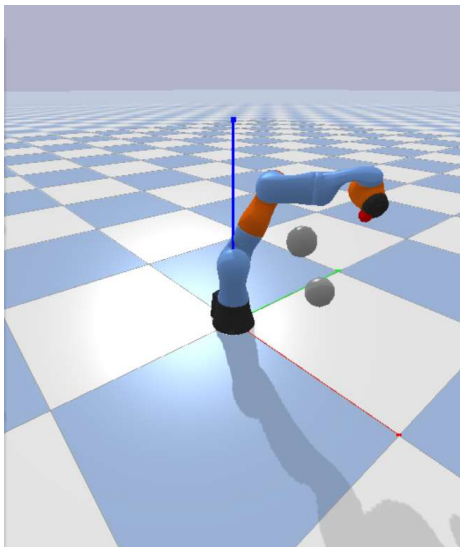


Robotic Arm Learning in Simulation

PPO Reaching and Demonstration-Assisted Manipulation with the KUKA LBR iiwa and the Franka Emika Panda



PyBullet simulations: KUKA LBR iiwa (left) and Franka Emika Panda (right)

Contributors

Bibek Karki
Suprem Shrestha
Sushant Shrestha
Utsav Rai

Institution

Birmingham City University

Date

6 October 2026

Abstract

Background. Simulation is the standard first step for developing robot-manipulation controllers, because failures are cheap and every trial can be seeded and repeated. This project asked how far reinforcement learning (RL) and demonstration-assisted learning can control simulated robotic arms in PyBullet, and how honestly such controllers can be evaluated.

Methods. In Phase 1, Proximal Policy Optimization (PPO) was trained to move the end effector of a 7-joint KUKA LBR iiwa arm to random three-dimensional targets within 5 cm. We ran a three-seed observation ablation, PPO and Soft Actor-Critic (SAC) experiments with one and two spherical obstacles, an equal-budget paired reward ablation, a failure analysis, and a comparison with an analytic inverse-kinematics (IK) baseline. In Phase 2, a Franka Emika Panda arm with a physical two-finger gripper was given a four-value Cartesian action interface (x, y, z increments and finger opening) executed through IK. Policies were trained by behaviour cloning and DAgger from a training-time feedback teacher, with optional short PPO fine-tuning. They were evaluated without the teacher on seeded held-out scenes, a 17-case stress suite, random diagonal layouts, 23/40/65 cm transfers and a persistent three-object scene.

Results. The relative-vector observation raised the three-seed mean final evaluation success from 34.4% to 60.0%, although seed variance was large. The selected KUKA reaching checkpoint reached 409 of 500 targets (81.8%; 95% Wilson interval 78.2–84.9%) against 500 of 500 for analytic IK. A dense progress-shaped reward did not beat the original distance reward in any of three paired continuation seeds (held-out mean 57.7% against 70.7%). Single-sphere obstacle avoidance reached 87.8% (sample SD 10.2 percentage points, $n = 3$ seeds). PPO from scratch with joint-space actions learned no complete Panda pick-and-place in two 1,000,000-step runs (0% success). After the interface redesign, the final shared Panda checkpoint passed 34 of 34 stress episodes, 20 of 20 fresh random diagonal layouts, 3 of 3 distance tasks and a 3 of 3 persistent three-object sequence, with typical placement errors near 1 cm. A 10,240-step PPO continuation of an intermediate checkpoint did not improve it (stress 33/34 to 32/34; mean diagonal error 11.1 mm to 18.5 mm).

Conclusions. Interface design, demonstrations and evaluation discipline mattered more than additional PPO steps. The results are finite, simulation-only and rely on exact simulator state, including the known object yaw, so they do not establish universal reliability, camera-based perception or hardware transfer.

Keywords: *reinforcement learning; proximal policy optimization; imitation learning; DAgger; robotic manipulation; pick and place; PyBullet; KUKA iiwa; Franka Emika Panda; reproducibility.*

Contents

1. Introduction	1
1.1 Background and motivation	1
1.2 Related work	1
1.3 Problem statement, research questions and objectives	2
1.4 Contributions and scope	3
1.5 Structure of the report	3
2. Methods	4
2.1 Development pathway and project organisation	4
2.2 Software environment and tooling	4
2.3 Simulation platform and robots	5
2.4 KUKA reaching and obstacle-avoidance environments	6
2.5 KUKA training and evaluation protocol	7
2.6 Panda pick-and-place environment	8
2.7 Environment profiles	11
2.8 Controllers and learning methods	12
2.9 Experimental design and evaluation protocol	13
2.10 Quality assurance and reproducibility	15
3. Results	16
3.1 Overview and reporting conventions	16
3.2 KUKA reaching	16
3.3 KUKA obstacle avoidance	21
3.4 Panda pick-and-place learned from scratch with PPO	22
3.5 Scripted physical-gripper baseline	23
3.6 Learned Panda controllers: from the nominal task to scene generalisation	24
3.7 Random diagonal layouts, 23/40/65 cm transfers and the persistent scene	27
3.8 The final shared Panda checkpoint	29
3.9 PPO continuation of an intermediate shared checkpoint	31
3.10 Overall summary of results	33
4. Discussion	35
4.1 Interpretation of the main findings	35
4.2 Why the redesigned Panda problem became tractable	36
4.3 Limitations and threats to validity	36
4.4 Implications and recommendations	37
4.5 Future work	38
5. Conclusion	39
References	40
Appendix A. Reproduction, evidence and supporting data	43
A.1 Setup and main commands	43
A.2 Development scripts of the early KUKA phase	45
A.3 Automated test inventory (42 tests)	46
A.4 Per-case stress-suite results across checkpoints	46
A.5 Checkpoint hashes (SHA-256)	48
A.6 Repository structure	48
A.7 Glossary	49

List of Figures

Figure 1. Development pathway of the project, from KUKA reaching experiments through the failed from-scratch Panda attempt to the shared imitation-trained checkpoint	4
Figure 2. KUKA LBR iiwa in PyBullet with two grey spherical obstacles and the red target sphere near the end effector	5
Figure 3. Franka Emika Panda in PyBullet during the pick-and-place task.....	6
Figure 4. Panda control loop and training pipeline	9
Figure 5. Observation ablation: success rate on the 30-episode in-training evaluation	17
Figure 6. First single-seed PPO run on random-target reaching (500,000 steps).....	18
Figure 7. Final distance to target for each of the 500 evaluation episodes of the selected PPO checkpoint	19
Figure 8. Selected PPO checkpoint: reached (green) and failed (orange) targets in the 500-episode final evaluation, plotted by target position.....	20
Figure 9. Equal-budget reward ablation: hold-out success of the original (legacy) reward and the shaped reward for each continuation seed	21
Figure 10. One-sphere obstacle avoidance with PPO and the relative-vector observation: mean in-training success over three seeds with the range across seeds shaded.....	22
Figure 11. Initial scripted-controller success over the mass $\times$ friction grid (54 trials).....	23
Figure 12. Four frames of a scripted pick-and-place episode: approach, closing the fingers around the cube, moving to the destination	24
Figure 13. Nominal Panda learning: development validation success (left) and the recorded action-fitting loss (right) over the imitation and DAgger rounds	25
Figure 14. Source (square) and destination (star) positions of the three fixed transfer tasks of 23, 40 and 65 cm; arrows are straight lines, not measured trajectories.....	28
Figure 15. Frames from the recording of the persistent three-object scene (tasks 1 and 3, before and after placement)	28
Figure 16. Successes out of two paired seeds for each of the 17 stress cases across six Panda checkpoints (columns).....	30
Figure 17. Final shared checkpoint: success (left) and mean placement error (right) in each of the 17 stress cases (two paired seeds per case).....	30
Figure 18. Final XY placement error of the 20 fresh random diagonal episodes (seeds from 56,000).....	31
Figure 19. Paired comparison before and after 10,240 PPO steps: stress success (left) and mean diagonal XY error (right).....	32
Figure 20. Validation success and mean reward during the 10,240-step PPO continuation (three recorded evaluations, joined for readability; single seed)	33
Figure 21. Success rates of the main evaluations with 95% Wilson intervals (counts on the right).....	33

List of Tables

Table 1. Software stack and pinned versions	5
Table 2. KUKA environment specifications	7
Table 3. KUKA training experiments	8
Table 4. Panda observation layout (24 values nominal; 32 values obstacle-aware)	9
Table 5. Panda reward components (reward version 3)	10
Table 6. Scene parameters, defaults and randomisation ranges	11
Table 7. Panda environment profiles	11
Table 8. Learning settings for the Panda policies	13
Table 9. Seed partitions (selected)	14
Table 10. Seventeen-case stress suite definitions (all other parameters at defaults)	15
Table 11. Observation ablation, per seed (PPO, 500,000 steps, 30 deterministic evaluation episodes every 5,000 steps)	17
Table 12. Final KUKA reaching evaluation on 500 unseen targets (seeds 2000–2499, success within 5 cm)	18
Table 13. Failure diagnostics for the selected KUKA PPO checkpoint (500 episodes)	19
Table 14. Reward ablation: success rate of each continuation run (200 test targets, seeds 0–199; 200 hold-out targets, seeds 1,000–1,199)	20
Table 15. KUKA obstacle-avoidance runs (30-episode in-training evaluations every 5,000 steps; relative-vector observation)	21
Table 16. Scripted baseline: success over the mass × friction grid (54 trials per run)	23
Table 17. Nominal Panda checkpoints (fixed 5 cm cube; behaviour cloning + DAgger + 1,024 PPO steps)	25
Table 18. Stress-suite results as the Panda controller gained capabilities	26
Table 19. Scene generalisation on the 17-case suite (two paired seeds per case, 34 episodes)	27
Table 20. Layout, distance and persistent-scene results for the specialist checkpoints	29
Table 21. Final shared checkpoint (panda_simulation_policy.zip, yaw_aligned_v3 profile)	31
Table 22. Paired comparison: intermediate imitation checkpoint against its 10,240-step PPO continuation (seed 54,000 onwards)	32
Table 23. Checkpoint inventory (SHA-256 truncated to 12 characters)	34
Table 24. Main command-line entry points (python -m robotic_arm ...)	43
Table 25. Scripts in the early (v2.8 “watch-precision”) archive	45
Table 26. Unit and integration tests (Python unittest; 42 in total)	46
Table 27. Successes per case (of 2) for six checkpoints and the final shared checkpoint’s mean placement error	47
Table 28. Full SHA-256 hashes of the evaluated checkpoints, taken from the saved summaries and model metadata	48

1. Introduction

1.1 Background and motivation

Robotic manipulation (reaching, grasping, carrying and placing objects) underpins applications from warehouse logistics to assistive robotics. Hand-designed controllers can solve narrow versions of these tasks, but they need careful re-engineering whenever the objects, the layout or the physics change. Learning-based control offers a different route: a policy is adapted from experience rather than programmed, and reinforcement learning (RL) is the best-known framework for doing so (Sutton and Barto, 2018). Surveys of RL in robotics stress that the field is hard for specific reasons: states and actions are continuous and high-dimensional, real-world trials are slow and risky, and reward functions are difficult to specify (Kober, Bagnell and Peters, 2013; Ibarz et al., 2021).

Physics simulation addresses part of this problem. In a simulator a failed grasp costs nothing, every trial can be seeded and replayed, and thousands of episodes can be generated cheaply. Simulation is also where claims about learned controllers are first checked, and its limits are well documented: transfer to real hardware depends on modelling error and on techniques such as domain randomisation (Tobin et al., 2017; Zhao, Queralta and Westerlund, 2020; Dulac-Arnold et al., 2021). This project therefore treats simulation as a controlled laboratory. Its purpose is to study the design and evaluation of learned controllers, not to claim real-world capability.

The project was carried out by a student group at Birmingham City University. It began with a deliberately simple benchmark, in which a 7-joint KUKA LBR iiwa arm learns to reach random targets with PPO (Schulman et al., 2017). It then progressed to a much harder problem, a Franka Emika Panda arm (Haddadin et al., 2022) that must grasp, lift, carry and place an object using a physical two-finger gripper. The two phases differ in difficulty and in what went wrong, and the contrast is the main source of the lessons reported here.

1.2 Related work

Policy-gradient RL and software. PPO optimises a clipped surrogate objective that limits how far each update moves the policy, giving stable learning at modest implementation cost (Schulman et al., 2017); it is normally paired with generalised advantage estimation (Schulman et al., 2016). Off-policy alternatives such as DDPG (Lillicrap et al., 2016) and Soft Actor-Critic (SAC; Haarnoja et al., 2018) reuse past experience and often learn faster on continuous control. We used the Stable-Baselines3 implementations (Raffin et al., 2021), the Gymnasium environment interface (Towers et al., 2024; originally Brockman et al., 2016) and the PyBullet physics engine (Coumans and Bai, 2016–2021).

Reward design and sparse tasks. Reward shaping can speed up learning, but only particular forms (potential-based shaping) are guaranteed to leave the optimal policy unchanged (Ng, Harada and Russell, 1999). Multi-stage manipulation is especially sensitive to reward design, because success is rare before the agent has learned to grasp. Hindsight experience replay (Andrychowicz et al., 2017) is one remedy for sparse goals. Our own experience, described in Section 3.4, shows the practical side of the problem: a plausible dense reward can be exploited, or can leave the agent in a different local optimum.

Learning from demonstrations. Imitation learning predates modern deep RL; the ALVINN system learned steering from a human driver’s examples (Pomerleau, 1989), and imitation has long been proposed as a route to humanoid skills (Schaal, 1999). Behaviour cloning, which is supervised learning on state–action pairs, suffers from compounding errors because the learner visits states the demonstrator never showed. DAgger addresses this by repeatedly querying the expert on states the learner visits and aggregating the data (Ross, Gordon and Bagnell, 2011); Hussein et al. (2017) survey the wider family. Demonstrations are also used to help RL overcome exploration problems (Nair et al., 2018), to initialise policy gradients for dexterous manipulation (Rajeswaran et al., 2018) and to combine imitation with RL for diverse visuomotor skills (Zhu et al., 2018). The Panda work in this report follows the same broad idea but is explicit that its final policies are imitation-trained.

Evaluation and reproducibility. Deep RL results are sensitive to seeds, implementation details and hyperparameters (Henderson et al., 2018; Engstrom et al., 2020), and a handful of runs rarely supports a confident ranking of methods; interval estimates and honest accounting of the number of runs are recommended (Agarwal et al., 2021). Binomial success rates from small test sets are also uncertain, which is why we report Wilson score intervals (Wilson, 1927). These ideas shaped the experimental protocol in Section 2.9.

Obstacle avoidance and perception. Classical manipulator obstacle avoidance uses potential fields and related geometric methods (Khatib, 1986). Our obstacle experiments instead give the learner exact obstacle geometry as part of its observation. End-to-end learning from camera images (Levine et al., 2016) is a natural next step but is outside the scope of this project.

1.3 Problem statement, research questions and objectives

The central problem is to determine how reliably RL and demonstration-assisted learning can control simulated arms for reaching and pick-and-place, and what experimental evidence justifies calling a controller “working”. The work addressed four research questions:

- RQ1.** Does adding an explicit relative target vector to the observation improve PPO reaching?
- RQ2.** Does a dense, progress-shaped reward improve on the original distance-based reward in an equal-budget, paired-seed comparison?
- RQ3.** Can PPO learn Panda pick-and-place from scratch with joint-space actions? If not, why not, and which changes make the problem tractable?
- RQ4.** How far does a demonstration-trained Cartesian policy generalise across object physics, obstacles, layouts and orientation, and does short PPO fine-tuning add anything?

The objectives were to:

- build a seeded, tested Gymnasium and PyBullet environment for KUKA reaching and train a PPO reaching agent (O1);
- quantify design choices (observation, algorithm, reward, obstacle count and success threshold) with controlled experiments (O2);
- attempt Panda pick-and-place with physical grasping and document the failure modes honestly (O3);

- develop a learned pick-and-place policy that is evaluated without any teacher on held-out seeds (O4);
- extend the policy to different object properties, obstacles, transfer distances, a persistent multi-object scene and object orientation (O5); and
- preserve the evidence (seeds, checkpoint hashes, immutable reports) so every reported number can be re-checked (O6).

1.4 Contributions and scope

The main contributions are: (i) a reproducible two-robot simulation codebase with 42 automated unit tests; (ii) controlled evidence on observation design, algorithm choice and reward shaping for KUKA reaching, including a negative result for dense shaping; (iii) a documented negative result for PPO from scratch on Panda pick-and-place; (iv) a Cartesian-action, IK-based interface and a teacher, behaviour-cloning and DAgger pipeline that yields policies passing a broad finite test suite; (v) a paired experiment showing that a short PPO continuation did not improve an imitation-trained checkpoint; and (vi) an explicit statement of what the results do not show.

Scope statement

Simulation only. No hardware was used and no real-world transfer is claimed.

Policies receive exact simulator state (object, goal and obstacle positions, and for the final model the initial object yaw through the low-level controller). No camera-based perception is learned; the rendered camera is used only to record media.

The final shared Panda checkpoint is imitation-trained. It is not a PPO-trained replacement for earlier work, and the report keeps the two clearly separated.

1.5 Structure of the report

This report follows the IMRaD structure. Section 2 (Methods) describes the software, the environments, the learning methods and the evaluation protocol. Section 3 (Results) reports the KUKA experiments, the failed from-scratch Panda attempt, and the progression of Panda controllers up to the shared checkpoint. Section 4 (Discussion) interprets the findings, relates them to the literature, and sets out limitations, threats to validity and future work. Section 5 gives the conclusions. The appendices list reproduction commands, constants, per-case results, checkpoint hashes and the test inventory.

2. Methods

2.1 Development pathway and project organisation

Work proceeded in two phases, summarised in Figure 1. Two code archives document it: an early backup (version 2.8, “watch-precision”) containing the KUKA work and a first Panda attempt, and the final repository containing the restructured Python package, the saved checkpoints, the evidence under `results/` and the unit tests. The early archive grew through small scripted checks: loading the arm and listing its joints, dropping a cube to test physics, applying a constant action to confirm that joint 0 moves by about 1 rad over 50 steps, printing the observation vector, confirming that the reward rises as the distance falls, running random-action episodes, and teleporting the arm to an IK solution to confirm that the success flag fires. Training and evaluation scripts followed, and finally the package, tests and reporting tools.

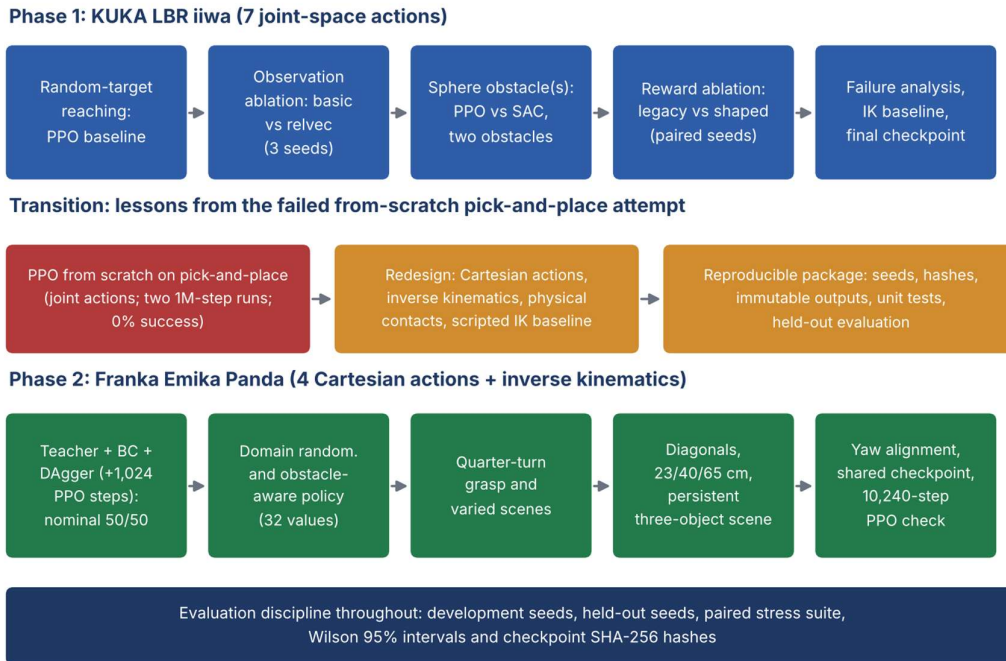


Figure 1. Development pathway of the project, from KUKA reaching experiments through the failed from-scratch Panda attempt to the shared imitation-trained checkpoint.

2.2 Software environment and tooling

Everything ran on CPU in Python 3.11 on Windows (PowerShell, Visual Studio Code), with dependency versions pinned in `requirements.txt` and recorded in `results/environment.json` (Table 1). Pinning matters because PyBullet physics, Stable-Baselines3 defaults and PyTorch numerics can change between releases and alter results (Henderson et al., 2018; Engstrom et al., 2020). Array computation used NumPy (Harris et al., 2020) and neural networks used PyTorch (Paszke et al., 2019).

Table 1. Software stack and pinned versions.

Component	Version	Role
Python	3.11 (3.11.9 interpreter)	Language and runtime
PyBullet	3.2.6	Rigid-body physics, URDF loading, inverse kinematics, GUI and tiny-renderer camera
Gymnasium	0.29.1	Environment API: reset, step, Box spaces
Stable-Baselines3	2.3.2	PPO and SAC implementations, evaluation callbacks, model serialisation
PyTorch	2.3.1 (CPU build)	Neural networks; behaviour-cloning optimiser
NumPy	1.26.4	Numerics and seeded random generators
Matplotlib	3.9.0	Plots
Pillow	12.3.0	GIF frame handling for recordings
PyYAML	6.0.2	Recorded in environment.json only

2.3 Simulation platform and robots

Both robots were simulated in PyBullet using the URDF models bundled with `pybullet_data`. Gravity was set to 9.8 m/s^2 for KUKA environments and 9.81 m/s^2 for Panda environments. The Panda environments set the physics step to $1/240 \text{ s}$ with 150 solver iterations and used the debug GUI only for observation and screen capture. Training and evaluation ran in headless (DIRECT) mode. Figure 2 and Figure 3 show the two simulated robots.

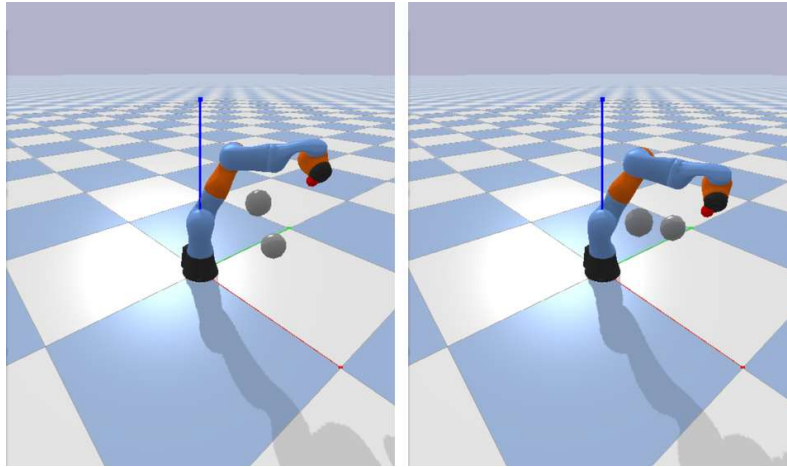


Figure 2. KUKA LBR iiwa in PyBullet with two grey spherical obstacles and the red target sphere near the end effector. The two frames are screenshots from the obstacle-avoidance evaluation (cropped to the simulation viewport).

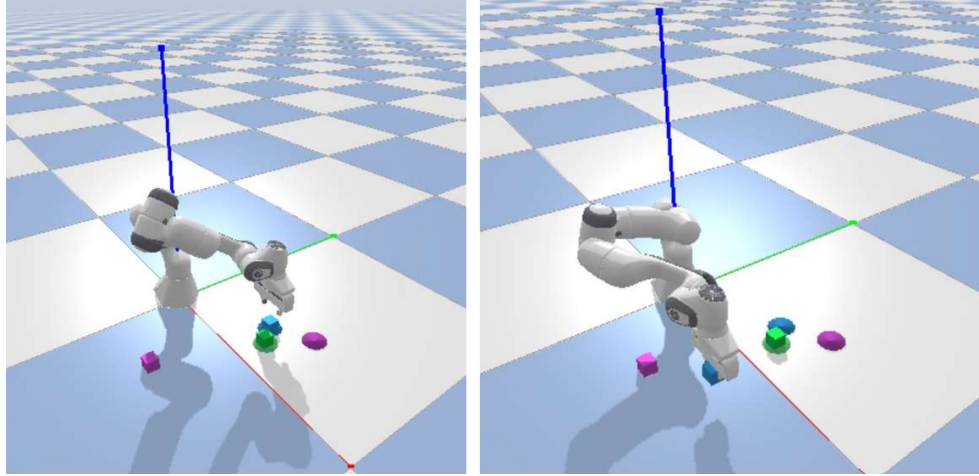


Figure 3. Franka Emika Panda in PyBullet during the pick-and-place task. Coloured objects are the persistent-scene cubes and the green disc marks the goal; the two frames show different stages of an episode (cropped screenshots).

KUKA LBR iiwa. A seven-joint revolute arm fixed at the origin. All seven joints were position-controlled, and the end effector was the last link (index 6). **Franka Emika Panda.** The Panda is a seven-degree-of-freedom collaborative arm widely used in research (Haddadin et al., 2022). The model has a two-finger parallel gripper. Joints 0–6 are the arm, joints 9 and 10 are the finger joints, and link 11 is the grasp target frame used for inverse kinematics. The arm was reset to a fixed home posture, and arm motor force limits were 87 N·m for joints 0–3 and 12 N·m for joints 4–6, with 10 N for each finger. Finger and object friction were set explicitly. Objects were grasped through contact and friction, with no attachment constraints and no in-episode teleportation of objects.

2.4 KUKA reaching and obstacle-avoidance environments

The reaching task asks the arm to bring its end effector within 5 cm of a randomly placed target sphere. Targets were sampled uniformly in a box in front of the arm (x 0.3–0.6 m, y –0.4–0.4 m, z 0.3–0.7 m). Each decision applied a bounded joint-angle increment (at most 0.05 rad per joint) as a position-control target and advanced the simulation by five physics steps. An episode ended on success or after 200 decisions. Table 2 lists the environment specification, including the early package (used for the ablations) and the cleaned-up final package (used for the final checkpoint).

Table 2. KUKA environment specifications.

Item	Early package (v2.8 backup)	Final package (robotic_arm)
Robot and control	KUKA iiwa, 7 joints; action = 7 values in $[-1, 1]$ scaled by 0.05 rad; position control, force 300; 5 physics sub-steps	Same; targets clipped to joint limits; explicit 1/240 s step; ground-contact detection
Observation	Joint positions (7), joint velocities (7), end-effector xyz (3), target xyz (3) = 20 values (<i>basic</i>); plus target – end-effector vector (3) = 23 values (<i>relvec</i>)	20 values (same as <i>basic</i>); relative vector dropped from the final environment
Target distribution	Uniform x 0.3–0.6, y –0.4–0.4, z 0.3–0.7 m	Identical (kept to allow fair checkpoint comparison); optional fixed target
Success / termination	Distance < 0.05 m (0.02 m in precision variant); terminate on success; truncate at 200 steps	Distance < 0.05 m; terminate on success; truncate at 200 steps
Reward	Legacy: –distance, +10 on success	<i>legacy</i> (as before) or <i>shaped</i> : $-d + 10(d_{i-1} - d_i) - 0.001\ a\ ^2 - 1[\text{ground contact}]$, +10 on success
One obstacle	Grey sphere, radius 0.08 m, sampled x 0.25–0.5, y –0.3–0.3, z 0.3–0.6 m, at least 0.15 m from target. Observation + obstacle xyz (23 <i>basic</i> / 26 <i>relvec</i>). Reward – d , +10 success, –5 and episode ends on collision	Not carried into the final package
Two obstacles	Two spheres, radius 0.07 m, at least 0.18 m apart and 0.15 m from the target. Observation 26 <i>basic</i> / 29 <i>relvec</i>	Not carried into the final package
Collision test	Only the end-effector point is tested against the obstacle sphere centre; arm links are not tested	Ground contact is flagged for the whole arm
Baselines	None	Analytic IK controller (PyBullet IK, bounded increments) and a random controller

The *shaped* reward adds a term proportional to the progress made towards the target at each step. Its form is close to potential-based shaping (Ng, Harada and Russell, 1999), but the action penalty and ground-contact penalty are not potential-based, and the discount factor in training is 0.99 rather than 1, so the usual invariance guarantee does not strictly apply. This matters when interpreting Section 3.2.5.

2.5 KUKA training and evaluation protocol

PPO (Schulman et al., 2017) used the Stable-Baselines3 `MlpPolicy` (two hidden layers of 64 units, default for this library) with `n_steps = 2048`, `batch_size = 64` and `n_epochs = 10`; all other hyperparameters were library defaults (learning rate 3×10^{-4} , $\gamma = 0.99$, GAE $\lambda = 0.95$, clip range 0.2, entropy coefficient 0). The stored reaching checkpoint confirms these values. SAC (Haarnoja et al., 2018) used library defaults except `learning_starts = 2000`, `train_freq = 1` and `gradient_steps = 1`. Table 3 lists the KUKA experiments.

Table 3. KUKA training experiments.

Experiment	Algorithm	Configuration	Evaluation during/after training
Observation ablation	PPO	<i>basic</i> vs <i>re/vec</i> ; seeds 0, 1, 2; 500,000 steps each	Callback every 5,000 steps: 30 deterministic episodes on a separate environment seeded <i>seed</i> + 1000
One-sphere avoidance	PPO	<i>re/vec</i> ; seeds 0, 1, 2; 500,000 steps; threshold 0.05 m	Same callback
Algorithm comparison	SAC	<i>re/vec</i> ; seed 0; 300,000 steps; one sphere	Same callback
Precision variant	PPO	Success threshold 0.02 m; seed 0; 800,000 steps; one sphere	Same callback
Two spheres	PPO	<i>re/vec</i> ; seed 0; 500,000 steps	Same callback
Reward ablation	PPO	Resume from reference checkpoint (450,000 steps); +100,000 steps; seeds 7, 19, 42; legacy vs shaped	Validation seeds from 10,000; 20 episodes every 10,000 steps; best by (success, -mean distance); then 200 test targets (seeds 0–199) and 200 hold-out targets (seeds 1000–1199)
Final evaluation	PPO, IK	Selected checkpoint, reference checkpoint and IK controller	500 unseen targets, seeds 2000–2499; 95% Wilson interval; ground-collision count

The reward ablation was designed as a paired comparison: both reward modes continued from the same starting checkpoint (SHA-256 recorded) for the same number of additional steps with the same three continuation seeds, then were evaluated on identical test and hold-out targets. The design is explicitly a continuation study, not training from scratch. The best checkpoint within each run was chosen on validation data, before looking at test or hold-out data, and the initial checkpoint was itself evaluated first so that a worse update could not replace it.

Two further analyses used the final evaluation episodes: a failure analysis (per-episode diagnostics: action saturation, closeness of joints to limits, and stationary distance, split by target height) and a plot of failures against target position. The analysis script labels its results as associations, not causes. A noise-robustness script was also written (Gaussian noise added to observations at standard deviations 0 to 0.1) but no saved noise results are included in the archives, so none are reported.

2.6 Panda pick-and-place environment

First attempt (joint-space). The early Panda environment had a table and a small cube, 29 observation values and eight actions (seven joint increments and one gripper command). It used a phase-based dense reward (distance to cube before grasp; progress towards the goal after grasp; a holding bonus; a drop penalty of 15) with a 300-step horizon. Its results are reported in Section 3.4.

Redesigned environment. The final environment replaced joint increments with a four-value Cartesian interface: the policy outputs (dx, dy, dz) in $[-1, 1]$ and a finger-opening command. The new end-effector target is the *measured* current position plus $1 \text{ cm} \times (\text{dx}, \text{dy}, \text{dz})$, clipped to the workspace $x \text{ } 0.25\text{--}0.65 \text{ m}$, $y \text{ } -0.35\text{--}0.35 \text{ m}$, $z \text{ } 0.035\text{--}0.50 \text{ m}$. Anchoring each increment to the measured pose prevents commands from accumulating behind a stalled arm. Inverse kinematics (link 11, fixed gripper orientation, up to 100 iterations) converts the target into seven joint motor commands, and the finger opening is $(a + 1) \times 0.02 \text{ m}$, so it spans $0\text{--}0.04$

m. Each decision advances 12 physics steps at 240 Hz, giving a 20 Hz control rate, and the horizon is 300 steps (500 in the obstacle-aware profiles). Figure 4 shows the loop.

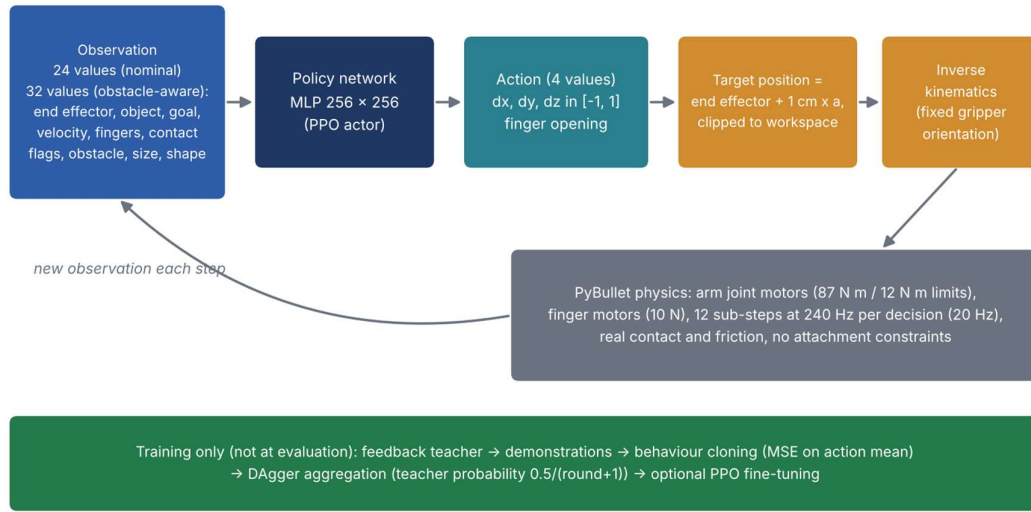


Figure 4. Panda control loop and training pipeline. The policy outputs four Cartesian actions that are converted to joint motor commands by inverse kinematics; the teacher is used only during training.

The observation (Table 4) contains the end-effector, object and goal positions, relative vectors, object velocity, finger positions, two history flags (grasped, lifted) and two current finger-contact flags. The obstacle-aware profiles append the obstacle centre, its half-extents, the object size and a cylinder flag. The goal height is the object’s resting centre height.

Table 4. Panda observation layout (24 values nominal; 32 values obstacle-aware).

Index	Content	Notes
0–2	End-effector position	Link 11, world frame (m)
3–5	Object position	Centre of mass
6–8	Goal position	Destination xy; z = half object size
9–11	Object – end effector	Relative vector
12–14	Goal – object	Relative vector
15–17	Object linear velocity	m/s
18–19	Finger joint positions	Joints 9 and 10 (m)
20–21	History flags: grasped, lifted	Set once both fingers touched; lifted = grasp and height > 0.15 m
22–23	Current contact flags (finger 9, finger 10)	From PyBullet contact points
24–26	Obstacle centre xyz	Zero if no obstacle (32-value profiles only)
27–29	Obstacle half-extents (x, y, z)	z = height/2
30–31	Object size; cylinder flag	Size in m; 1 for cylinders

2.6.1 Reward and success definition

The reward (version 3, Table 5) pays new achievements only, so that oscillating contact or height cannot farm reward. Success requires a physically meaningful sequence: a prior both-finger grasp, a lift above 15 cm while held, release, a settled object within 5 cm (horizontal) of the goal, and ten consecutive valid steps. “Settled” means the object’s centre is within 1.5 cm of its resting height and its speed is below 0.03 m/s; “released” means no gripper–object contact and fingers open beyond 2.5 cm. In the obstacle-aware profiles any contact between the arm or the object and the obstacle ends the episode as a failure with a -20 reward. No success flag is ever forced. In the lift stage a ground slide of the object beyond 8 cm also terminates the episode.

Table 5. Panda reward components (reward version 3).

Component	Value
Step cost and action penalty	$-0.05 - 0.01\ a\ ^2$ per decision
Progress towards object	$+10 \times$ reduction in best-ever end-effector–object distance (new best only)
Sliding penalty (lift and place stages)	$-40 \times$ horizontal object displacement per step while the object is below 6 cm
First two-finger grasp / loss of grasp	$+1$ once per episode / -1 each time the grasp is lost
Held height	$+100 \times$ increase in maximum held height (capped at 0.18 m); $+0.2 \times \min((z - 0.06)/0.09, 1)$ while held above 6 cm
First lift above 15 cm	$+5$
Placement progress (place stage, after lift)	$+15 \times$ reduction in horizontal placement error
Success	$+50$ after ten consecutive valid steps
Ground contact by the arm	-0.5 per step after the first such contact
Obstacle contact (obstacle-aware profiles)	-20 and the episode terminates as a failure
Lift stage: slide-out	-5 and termination when the object slides more than 8 cm on the ground

Three training stages were defined: *reach* (end effector within 4 cm of the object for ten steps), *lift* (held above 15 cm for ten steps) and *place* (the full success definition above).

2.6.2 Scene parameters and randomisation

Objects were boxes or cylinders of size 4–6 cm, mass 0.04–0.15 kg and friction 0.5–2.0. The nominal object was a 5 cm box, 0.08 kg, friction 2.0. A fixed box obstacle (height 0 to 0.24 m, half-widths 2.5–4.5 cm) could be placed at the midpoint between source and destination. Table 6 lists the parameters, defaults and randomisation ranges. Domain randomisation varies physics and size; scene randomisation varies layout, orientation and obstacle; and a cross-workspace sampler creates “diagonal” transfers between opposite regions of the table, with at least 10 cm of sideways separation and at least 31 cm of straight-line distance.

Table 6. Scene parameters, defaults and randomisation ranges.

Parameter	Default	Randomisation	Validation limits
Object mass	0.08 kg	$U(0.04, 0.15)$ (domain rand.)	$0 < m \leq 1$ kg
Friction (object and fingers)	2.0	$U(0.5, 2.0)$ (domain rand.)	≥ 0
Object size	0.05 m	$U(0.04, 0.06)$ (domain rand.); obstacle-aware curriculum cycles 4/5/6 cm by seed	0.02–0.075 m
Object shape	box	box or cylinder (domain rand.)	box or cylinder
Obstacle height	0 (none)	$\{0, 0.08, 0.16, 0.24\}$ m (scene rand.); curriculum $\{0, 0.08, 0.24\}$	0–0.4 m
Object yaw	0 rad	$U(-\pi, \pi)$ (scene rand.)	finite
Source xy	$(0.45, -0.15) + U(\pm 0.04, \pm 0.03)$	$U([0.36, -0.22], [0.54, -0.10])$ (scene rand.)	x 0.25–0.60, y -0.30–0.30
Destination xy	$(0.45, 0.20) + U([-0.04, -0.04], [0.04, 0.02])$	$U([0.36, 0.12], [0.54, 0.26])$ (scene rand.)	x 0.25–0.60, y -0.30–0.30
Obstacle offset	(0, 0)	$U(\pm 0.025)$ per axis (scene rand.)	$ \text{offset} \leq 0.10$ m
Obstacle half-widths	(0.04, 0.04)	$U(0.025, 0.045)$ (scene rand.)	0.01–0.08 m
Diagonal layouts	off	Source x 0.35–0.40 or 0.50–0.55, y -0.23 to -0.14; destination on the opposite x band, y 0.16–0.26	—

2.7 Environment profiles

As capabilities grew, the environment gained several profiles. Each saved checkpoint records its profile so that evaluation automatically selects a compatible environment (Table 7).

Table 7. Panda environment profiles.

Profile	Obs.	Description
nominal	24	Gripper pointing down; home joints $[0, -0.4, 0, -2, 0, 1.6, 0.8]$; 300-step horizon; no obstacle information
obstacle_aware_v1	32	Adds obstacle geometry and object size; new home joints $[-0.5, -0.4, 1.5, -2, 0, 1.6, 0.8]$; 500 steps; obstacle contact fails the episode
quarter_turn_v2	32	Gripper rotated by 90° about the vertical axis (Euler $[\pi, 0, \pi/2]$, wrist joint reset to 2.37 rad) so the jaws straddle the object alongside the obstacle; teacher releases up to 7 cm above the support and opens fully before withdrawing; four derived predicates (ready-to-release, fingers open, both contacts, clearance) appended to the network input
yaw_aligned_v3	32	Gripper yaw = $\pi/2$ + initial object yaw, wrapped and chosen to respect the wrist-joint limit (± 2.8973 rad); the policy does not choose or infer yaw
persistent	32	Three objects at fixed 23/40/65 cm layouts exist from the start; the active object and goal change after each success; the world and arm are not reset; earlier objects remain and are not part of the observation

2.8 Controllers and learning methods

2.8.1 Scripted IK baseline

A scripted controller (no learning) performs pick-and-place with the same physical gripper: settle, approach 25 cm above the source, descend to 3.5 cm, close the fingers for 240 steps, lift to 28 cm, move to the destination at 28 cm, descend, open the fingers, and withdraw.

Motion between waypoints uses smooth-step interpolation and IK. The controller checks for a both-finger grasp and a lift above 15 cm and reports a precise failure reason. It served both as a sanity check of the physics and as the origin of a benchmark over a mass $\times$ friction grid.

2.8.2 Training-time teacher

Learning used a hand-written feedback teacher that exists only during training. It computes a target position and a gripper command and converts them into the same four-value action space: `action = clip((target - end effector)/0.01, -1, 1)`. The teacher approaches above the object (at least 8 cm high) until the horizontal error is under 8 mm, descends to a grasp height (the larger of 3.5 cm or half the object size plus 1 cm), closes when low, lifts to a transport height (21 cm; 34 cm in later refinements), carries to the goal, descends, releases and withdraws. It can recover an accidental drop by re-grasping, and in obstacle scenes it first rises to a clearance of at least $2 \times$ obstacle half-height + 10 cm and routes around the obstacle. Episodes in which the teacher failed were discarded when collecting initial demonstrations. The teacher is never used when evaluating a policy.

2.8.3 Behaviour cloning, DAgger and PPO

Behaviour cloning minimised the mean-squared error between the policy's mean action and the teacher's action (Adam, learning rate 10^{-3} , mini-batches of 256, gradient clipping at 1.0), after which the policy's log standard deviation was fixed at -2.5 so that evaluation stays close to the mean action. DAgger (Ross, Gordon and Bagnell, 2011) then ran rounds in which the current policy drives the arm (mixed with the teacher with probability $0.5/(\text{round} + 1)$), the teacher labels every visited state, and the aggregated data are refit. The network was an MLP with two hidden layers of 256 units. Observations were standardised with a fixed mean and scale computed from the demonstrations and stored inside the checkpoint, so that evaluation applies the identical transform. Optional PPO fine-tuning used `n_steps = 1024`, batch size 64, learning rate 3×10^{-5} and a target KL of 0.01. Table 8 collects the settings.

Table 8. Learning settings for the Panda policies.

Setting	Value
Policy network	MLP 256 × 256 (actor and critic); fixed observation standardisation inside the checkpoint
Behaviour cloning	Adam, lr 1e-3, batch 256, MSE on action mean, grad-clip 1.0; epochs 200 default (150 in the nominal run; 100 in refinements); log-std set to -2.5 afterwards
DAGger	Default 3 rounds (nominal run: 2 rounds); 30 rollouts per round (nominal: 20 initial demonstrations); teacher probability 0.5/(round + 1); rounds saved as <code>imitation_round_k</code>
PPO fine-tuning	n_steps 1024, batch 64, lr 3e-5 (or 1e-5 in the final experiment), target KL 0.01 (0.001 in the final experiment), entropy coefficient 0
Model selection	Best development validation score before any hold-out look; ties keep the earliest checkpoint
Focused refinement	10 demonstrations per condition per round, 2 rounds, learner takes control 75% of steps after round 0 (teacher probability 0.25), transport height 0.34 m, added data duplicated twice, validation 5 seeds per condition, selection by weakest condition then total
Shared policy	Balanced behaviour cloning: 6,000 samples drawn from each of four saved datasets (scene generalisation 84,966; diagonal 25,460; distance 20,720; persistent 680 samples) plus yaw/size/obstacle demonstrations; 100 epochs; seed 49000; no PPO
Yaw refinement demonstrations	10 conditions × 5 teacher rollouts at seeds 52000–52004 (box sizes 4 and 6 cm at yaw 0.4, 0.785398, 1.2 and -0.785398 rad; a 6 cm box and a cylinder behind a 24 cm obstacle); all 50 succeeded; final dataset 51,682 samples

Because the final policies are imitation-trained, the report uses the terms accordingly: “PPO-trained” refers only to the KUKA reaching checkpoint and to the earlier nominal Panda checkpoint, which received 1,024 PPO fine-tuning steps after imitation; the more recent distance, obstacle and shared checkpoints are imitation-trained and are not claimed as RL results.

2.8.4 Focused refinement and the persistent scene

Each new capability was added as a focused refinement of the previous checkpoint: teacher demonstrations were collected for the new condition, the aggregated dataset was refit, and the result was validated on small per-condition sets before any hold-out run. The persistent-scene variant used the same four-value interface with a learned transition: after a task succeeded the environment switched the active object and goal without resetting the arm or the scene, and the sequence ended at the first failure. Finally, the shared policy merged the saved datasets with balanced sampling to avoid catastrophic forgetting of earlier capabilities.

2.9 Experimental design and evaluation protocol

Every evaluation used deterministic policy actions, fixed seeds and a frozen checkpoint, and wrote a summary and per-episode report that includes the model’s SHA-256 hash. Evaluation, training and recording commands refuse to overwrite existing output directories, which preserves failed as well as successful evidence. Seeds were partitioned by purpose (Table 9): demonstrations, development validation, hold-out sets and final tests used disjoint ranges, and a hold-out set was only used after the checkpoint had been selected. Binomial success rates are reported with Wilson 95% intervals (Wilson, 1927), because a result such as 20/20 still leaves considerable uncertainty about the true success probability.

Table 9. Seed partitions (selected).

Purpose	Seeds
KUKA training seeds	0, 1, 2 (evaluation environments use seed + 1000); continuation seeds 7, 19, 42; smoke tests with seed 99 ran a single 5,000-step evaluation and are not used
KUKA validation / test / hold-out / final	10,000+ / 0–199 / 1,000–1,199 / 2,000–2,499
Panda nominal	Demonstrations from seed 42 (20); DAgger from seed 100; validation 10,000–10,009; hold-out 5,000–5,049; lifting 3,000–3,049 (near cube) and 4,000–4,049 (full start)
Domain-randomised stress	Development 6,000; later checks 8,000 (three paired seeds per case, 10 cases)
Obstacle-aware v1	Development 9,000; hold-out 11,000 (36 episodes, 12 cases)
Quarter-turn v2	Validation 23,000–23,004; regression 2,000–2,019; hold-out 24,000–24,049 (6 cm box, 24 cm obstacle); stress 25,000–25,002
Varied scenes	Training 30,000–30,009; validation 31,000–31,004; hold-out 32,000–32,001 (17 cases); yaw-aligned candidate development 33,000 and hold-out 34,000
Diagonal / distance	Diagonal training 40,000–40,024, validation 41,000–41,004, replay 622,849, hold-out 42,000–42,019; distance training 43,000–43,009, validation 44,000–44,004, hold-out 45,000–45,019
Persistent scene	Training 46,000+; learned-policy check 47,000; evaluation 48,000
Shared policy	Training seed 49,000; yaw demonstrations 52,000–52,004; PPO experiment 54,000; final diagonal 56,000–56,019; final stress 56,000–56,001; distance tasks seed 2,000

The **stress suite** (Table 10) varies one or more physical factors at a time on paired target seeds (the same two seeds for every case), so that differences between cases reflect the condition rather than the layout. The original ten-case suite was extended to twelve cases (two additional obstacle cases) and then to seventeen (adding wide positions, rotated boxes and a shifted obstacle). It is a finite probe and not a random sample of scenes.

Table 10. Seventeen-case stress suite definitions (all other parameters at defaults).

Case	Variation
reference	Defaults: 5 cm box, 0.08 kg, friction 2.0, no obstacle
light / heavy	Mass 0.04 kg / 0.15 kg
slippery	Friction 0.5
small / large	Size 4 cm / 6 cm
cylinder	Cylindrical object
low_obstacle / blocking_obstacle	Obstacle height 8 cm / 24 cm between source and destination
heavy_slippery	Mass 0.15 kg and friction 0.5
large_blocking_obstacle	6 cm box with a 24 cm obstacle
cylinder_blocking_obstacle	Cylinder with a 24 cm obstacle
wide_left / wide_right	Source (0.36, -0.22), destination (0.38, 0.25) / source (0.53, -0.20), destination (0.52, 0.24)
rotated_small_box / rotated_large_box	4 cm / 6 cm box rotated by 45° (0.785398 rad)
shifted_obstacle	16 cm obstacle offset (-0.025, 0.02) m with half-widths (0.03, 0.045) m

2.10 Quality assurance and reproducibility

The package includes 42 automated unit tests (Table 26 in Appendix A), covering Gymnasium space and API contracts, seeded scene reproducibility, action and joint limits, physical grasps, lifts and releases, disconnect handling when the GUI is closed, protection of existing reports, persistent state transitions, camera rendering, metadata integrity and model-hash checks. An integration script (`scripts/verify_training.py`) runs a fresh and a resumed training job for 2,048 steps and checks that resumption adds steps and that validation artefacts exist. A dependency script prints the installed versions. The documentation states that passing the software tests does not guarantee that every policy rollout succeeds. This project involved no human participants, personal data or hardware; the only ethical considerations were academic integrity and honest reporting.

3. Results

3.1 Overview and reporting conventions

The results are reported in the order in which the work was done. Section 3.2 covers KUKA reaching (observation ablation, the selected checkpoint compared with an analytic baseline, a failure analysis and the reward ablation), and Section 3.3 covers KUKA obstacle avoidance. Section 3.4 reports the failed from-scratch Panda attempts and Section 3.5 the scripted Panda baseline that showed the physics was sound. Sections 3.6 to 3.9 follow the learned Panda controllers from the nominal task to the final shared checkpoint and the PPO continuation experiment. Section 3.10 gives an overall summary.

Three conventions apply throughout. First, a “success” is always the success definition of the relevant environment (Section 2.4 for KUKA, Section 2.6.1 for Panda); no success flag was ever forced. Second, results are labelled by the data on which they were measured: *in-training evaluation* (a small fixed set used to choose checkpoints, and therefore optimistic), *development* data (used while designing a method), and *hold-out* or *final* data (seeds untouched until the checkpoint was frozen). Third, proportions from small samples are accompanied by 95% Wilson intervals, which for 20/20 still extend down to 83.9%, and for 3/3 down to 43.9%.

3.2 KUKA reaching

3.2.1 Observation ablation

Both observation designs learned to reach, but with large variation between seeds (Figure 5, Table 11). After 500,000 steps, the three *basic* runs ended at 16.7%, 63.3% and 23.3% success on the 30-episode in-training evaluation (mean 34.4%, sample SD 25.2 percentage points), and the three *relvec* runs ended at 53.3%, 36.7% and 90.0% (mean 60.0%, SD 27.3). The best evaluation reached during training averaged 67.8% for *basic* and 83.3% for *relvec*. In every run the first non-zero evaluation came at 50,000 steps or later (*basic*: 80,000, 50,000 and 70,000 steps; *relvec*: 145,000, 115,000 and 50,000 steps), so the relative vector did not speed up the first successes. The success curves also oscillate strongly from one evaluation to the next, so a single evaluation point says little about the policy’s quality.

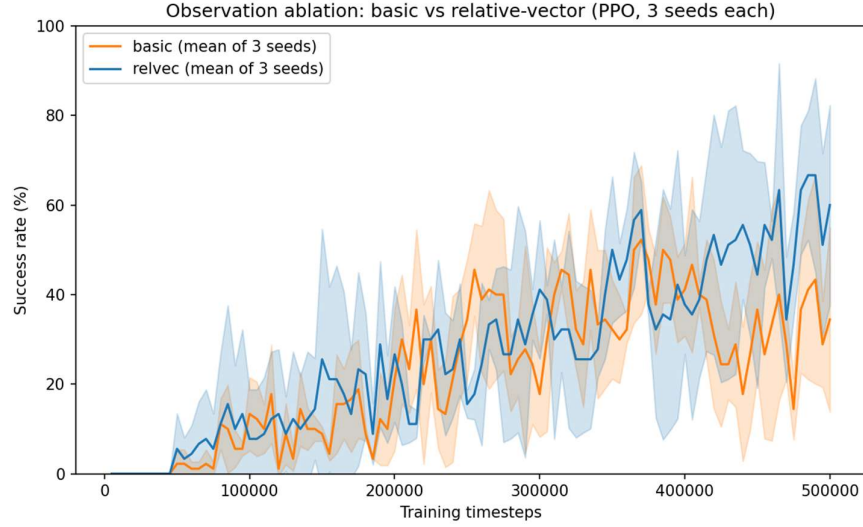


Figure 5. Observation ablation: success rate on the 30-episode in-training evaluation, averaged over three seeds for the basic (20-value) and relative-vector (23-value) observations. Shaded bands show the range across seeds.

Table 11. Observation ablation, per seed (PPO, 500,000 steps, 30 deterministic evaluation episodes every 5,000 steps).

Observation	Seed	Final eval. success	Best eval. success	Step of best	Mean of last 5 evals
basic (20)	0	16.7%	56.7%	370,000	18.0%
basic (20)	1	63.3%	73.3%	465,000	58.7%
basic (20)	2	23.3%	73.3%	405,000	34.0%
basic mean		34.4%	67.8%		36.9%
relvec (23)	0	53.3%	70.0%	350,000	51.3%
relvec (23)	1	36.7%	83.3%	465,000	48.0%
relvec (23)	2	90.0%	96.7%	490,000	85.3%
relvec mean		60.0%	83.3%		61.5%

Source: per-run evaluation logs (evaluations.npz) in the early archive. These are in-training evaluations, not held-out tests.

The relative-vector observation therefore raised the three-seed mean final evaluation success by 25.6 percentage points (from 34.4% to 60.0%). The direction agrees with the intuition that handing the network the vector it would otherwise have to compute makes learning easier, but the evidence is weak: a Welch test on the final values gives $t \approx 1.2$ on about 4 degrees of freedom, which is far from significant, and one *relvec* seed (seed 1) ended below the best *basic* seed. The result is best read as suggestive and as motivation for further seeds, which answers RQ1 only provisionally.

3.2.2 Selected reaching checkpoint, reference checkpoint and analytic baseline

Figure 6 shows the learning curve of the first single-seed PPO reaching run (500,000 steps, evaluated every 5,000 steps on 30 episodes). Its in-training success reaches 100% and its reward plateaus near -25 after roughly 400,000 steps. These numbers must be read with care. The in-training evaluation used a small fixed set of targets that was also used to pick the

“best” checkpoint, so it is biased upward. The archive keeps three early checkpoints whose folder names carry such in-training scores (46%, 48% and 99%). Evaluated on 500 fresh targets, the reference checkpoint from which later work continued succeeded on only 296 (59.2%).

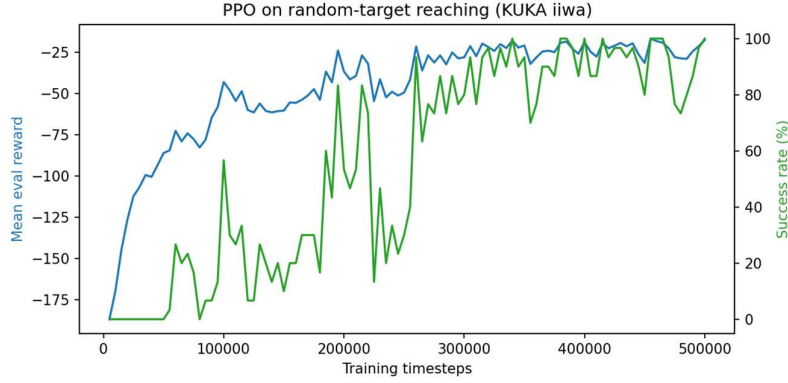


Figure 6. First single-seed PPO run on random-target reaching (500,000 steps, 30-episode evaluations): mean evaluation reward (left axis) and in-training success rate (right axis) against training steps. In-training evaluation is optimistic (small fixed target set).

The final evaluation used 500 unseen targets (seeds 2000–2499) and the same targets for every controller (Table 12). The selected PPO checkpoint reached 409/500 targets (81.8%, 95% Wilson interval 78.2–84.9%), against 296/500 (59.2%, 54.8–63.4%) for the reference checkpoint and 500/500 (99.2–100.0%) for the analytic IK controller. The two PPO intervals do not overlap, so the continuation experiment produced a genuine improvement over its starting point, but the learned policy remains well below the classical baseline. It is also slower on average (95 against 73 steps). No evaluation episode in any of the three sets touched the ground.

Table 12. Final KUKA reaching evaluation on 500 unseen targets (seeds 2000–2499, success within 5 cm).

Controller	Successes	95% Wilson interval	Mean final distance	Mean steps	Ground-collision episodes
Analytic IK baseline	500/500	99.2–100.0%	4.63 cm	73.1	0
Selected PPO checkpoint (reaching_policy)	409/500 (81.8%)	78.2–84.9%	9.83 cm	95.1	0
Reference PPO checkpoint (start of continuation)	296/500 (59.2%)	54.8–63.4%	26.3 cm	127.5	0

The PPO checkpoint is the legacy-reward continuation with seed 19, selected on validation seeds (10,000 onwards) before this evaluation.

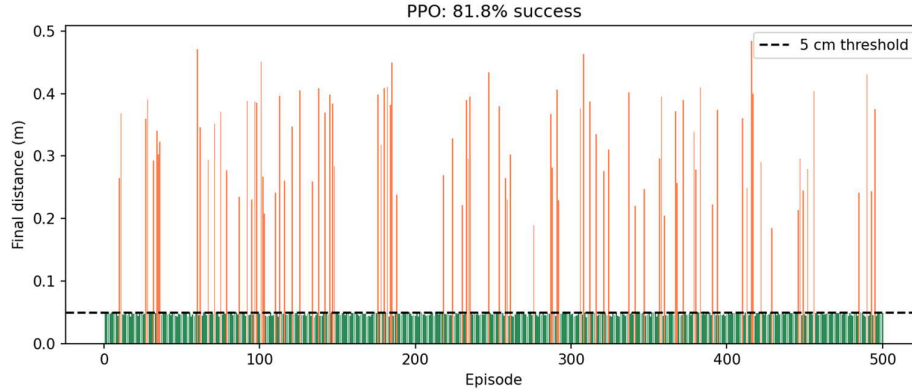


Figure 7. Final distance to target for each of the 500 evaluation episodes of the selected PPO checkpoint. Green bars are successes (below 5 cm); orange bars are failures.

The distance plot shows that the outcome is bimodal. Successful episodes end just under the threshold, while failures typically end 20–48 cm away. Failures are therefore not near-misses that a slightly larger tolerance would convert, which is why the mean final distance of the PPO policy (9.8 cm) is more than double the IK value although only 18% of episodes fail.

3.2.3 Failure analysis

The failure analysis of the 500 final episodes (Table 13) found 91 failures and no ground contact. Success did not depend strongly on target height (80.0% for targets below 0.5 m and 83.6% at or above). The clearest association was with joint limits. In failed episodes the arm spent on average 50.5% of its steps with at least one joint near its limit, against 1.1% in successful episodes, and failed episodes also spent more of their time stationary (1.9% against 0.05% of steps). The 3D target plot (Figure 8) shows failures concentrated at small x (targets close to the base), where the required joint configurations are awkward. Action saturation was 100% in both groups, meaning that the policy almost always commands the full ± 0.05 rad step on every joint, so it behaves as a bang-bang controller rather than making fine corrections. The analysis script states that these are associations and not proof of cause.

Table 13. Failure diagnostics for the selected KUKA PPO checkpoint (500 episodes).

Diagnostic	Successful episodes	Failed episodes
Episodes	409	91
Fraction of steps with a joint near its limit	1.12%	50.45%
Fraction of steps stationary (distance not changing)	0.05%	1.91%
Action saturation fraction	100%	100%
Episodes with ground contact	0	0
Success rate, target $z < 0.5$ m ($n = 250$)	80.0%	—
Success rate, target $z \geq 0.5$ m ($n = 250$)	83.6%	—

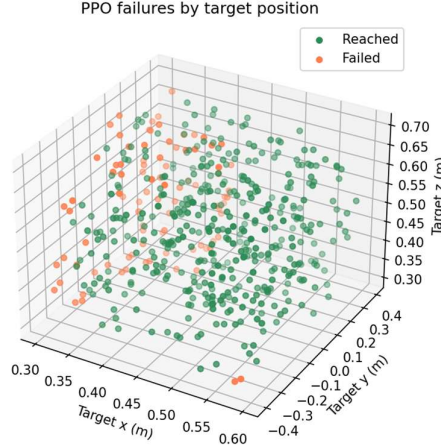


Figure 8. Selected PPO checkpoint: reached (green) and failed (orange) targets in the 500-episode final evaluation, plotted by target position.

3.2.4 Reward ablation: original distance reward against dense shaping

The paired continuation experiment (Section 2.5) compared the original reward with the progress-shaped reward on identical test (200 targets) and hold-out (200 targets) sets (Table 14, Figure 9). The shaped reward scored lower than the original reward for every one of the three continuation seeds, on both sets. On the hold-out targets the mean success was 70.7% for the original reward and 57.7% for the shaped reward, a paired difference of -8.5 , -20.5 and -10.0 percentage points for seeds 7, 19 and 42 (mean -13.0). Because the three seeds produce the same sign, the result is consistent, but with $n = 3$ and seed-to-seed SDs of 6–11 percentage points it is not strong enough for a general claim about shaping. It does show that adding a plausible dense term did not help in this setting, and RQ2 is answered negatively. The shaped runs also logged three ground-contact episodes on the test set (seed 42) even though ground contact was penalised, and the legacy seed-42 run logged one on test and two on hold-out.

Table 14. Reward ablation: success rate of each continuation run (200 test targets, seeds 0–199; 200 hold-out targets, seeds 1,000–1,199).

Seed	Legacy test	Legacy hold-out	Shaped test	Shaped hold-out	Hold-out difference (shaped – legacy)
7	64.5% (129/200)	58.5% (117/200)	55.5% (111/200)	50.0% (100/200)	-8.5 pp
19	81.5% (163/200)	76.0% (152/200)	61.0% (122/200)	55.5% (111/200)	-20.5 pp
42	80.0% (160/200)	77.5% (155/200)	67.5% (135/200)	67.5% (135/200)	-10.0 pp
Mean	75.3%	70.7%	61.3%	57.7%	-13.0 pp
Sample SD	9.4 pp	10.6 pp	6.0 pp	8.9 pp	

Starting checkpoint (reference): 59.2% on the 500 final targets. All runs add 100,000 steps from the same SHA-256-identified checkpoint.

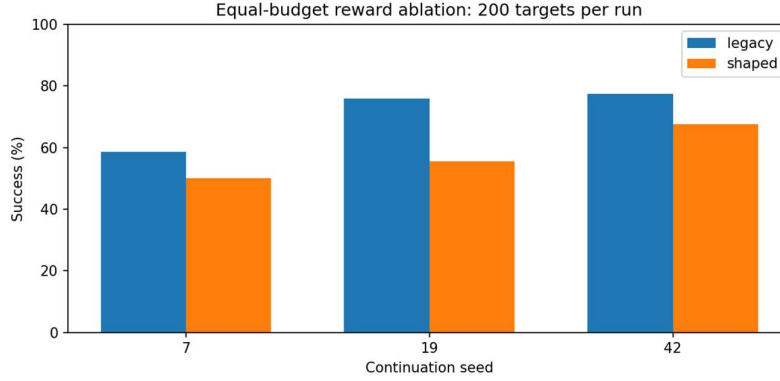


Figure 9. Equal-budget reward ablation: hold-out success of the original (legacy) reward and the shaped reward for each continuation seed.

Against the starting checkpoint’s 59.2% (measured on the separate 500 final targets), the legacy runs ended between 58.5% and 81.5% and the shaped runs between 50.0% and 67.5%, so extra PPO steps did not reliably help and the shaped seed 7 run ended below its starting point; only the legacy seed 19 and seed 42 runs reached the high-70% range. The legacy seed 19 checkpoint won the validation-based selection (validation success 80.0%, mean distance 9.65 cm) and became the shipped reaching policy.

3.3 KUKA obstacle avoidance

In the obstacle task the end effector must reach the target without touching a grey sphere (a collision ends the episode with a -5 reward). PPO with the relative-vector observation learned this reliably on one sphere (Table 15, Figure 10). The three seeds ended at 90.0%, 76.7% and 96.7% in-training evaluation success (mean 87.8%, sample SD 10.2 pp), and every seed touched 100% at some evaluation. SAC reached 100% final success on one seed after only 300,000 steps. Its first non-zero evaluation came at 25,000 steps (50,000–65,000 for the PPO seeds), its best value was already reached at 80,000 steps, and it averaged 99.3% over its last five evaluations. This is a single seed, so it cannot establish that SAC is better, but the sample efficiency is notable and agrees with the general reputation of off-policy methods on continuous control (Haarnoja et al., 2018).

Table 15. KUKA obstacle-avoidance runs (30-episode in-training evaluations every 5,000 steps; relative-vector observation).

Configuration	Seed	Steps	Final eval. success	Best (step)	Mean of last 5 evals
One sphere, PPO	0	500,000	90.0%	100% (235k)	94.7%
One sphere, PPO	1	500,000	76.7%	100% (305k)	86.0%
One sphere, PPO	2	500,000	96.7%	100% (480k)	95.3%
One sphere, PPO, mean			87.8%	100%	92.0%
One sphere, SAC	0	300,000	100%	100% (80k)	99.3%
One sphere, PPO, precision threshold 2 cm	0	800,000	73.3%	80.0% (595k)	39.3%
Two spheres, PPO	0	500,000	90.0%	100% (480k)	79.3%

These are in-training evaluation results with a small fixed target set; the last-five mean is the more representative figure because single evaluations fluctuate.

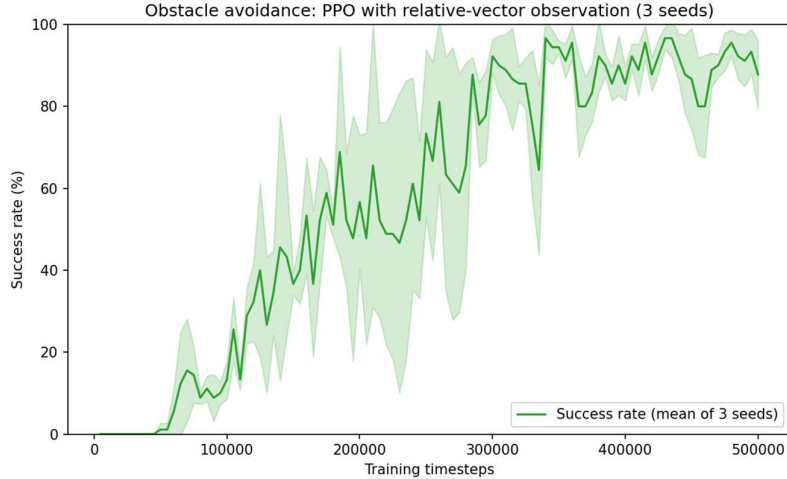


Figure 10. One-sphere obstacle avoidance with PPO and the relative-vector observation: mean in-training success over three seeds with the range across seeds shaded.

Two harder variants were less successful. Tightening the success threshold from 5 cm to 2 cm lowered the final success to 73.3% and, more importantly, the mean of the last five evaluations to 39.3%, which shows unstable late training: the 800,000-step run reached 80% once and then fluctuated. The two-sphere task reached 100% at its best point but averaged 79.3% over the last five evaluations. Two limits of the avoidance environment apply to all of these numbers: the collision test applies only to the end-effector point against each sphere, so the arm’s links are not checked, and the archives contain no held-out evaluation results for the obstacle policies. The obstacle experiments therefore show learnability on the training distribution and not safe avoidance.

3.4 Panda pick-and-place learned from scratch with PPO

After the KUKA work, the group attempted full pick-and-place with the Panda arm, with the joint-space interface, 29-value observation and phase-based dense reward of Section 2.6. Two PPO runs of 1,000,000 steps each were made (seeds 0 and 1, differing in their reward shaping). Neither produced a single successful pick-and-place in any periodic evaluation (20 episodes every 10,000 steps): the success rate was 0% at every evaluation of both runs, with a mean evaluation episode length of about 214 steps in run 0 and the full 300-step horizon in run 1, and a mean evaluation reward of about -36 and -34 .

The group’s own diagnosis of the two runs is informative. In the first run the agent learned to grasp the cube in roughly 68% of episodes but then always dropped it before placing, because the first reward made dropping early cheaper than carrying. After the reward was repaired the second run found a different local optimum: about 98% of its episodes never grasped at all. This is a familiar pattern for multi-stage manipulation: each phase has its own reward, and exploration rarely chains the phases into a full success. By contrast, a hand-written IK script performed the same task reliably, closing the fingers on the cube, lifting, carrying and placing it within 2 cm of the goal, which showed that the physics, contact detection and environment logic were sound and that the failure was one of learning, not of simulation. RQ3 is therefore answered: PPO from scratch with joint-space actions did not solve the task in 2,000,000 steps

of experience, and the remainder of the project changed the problem instead of simply training for longer.

3.5 Scripted physical-gripper baseline

The scripted Panda controller of Section 2.8.1 was evaluated on a grid of three masses (0.04, 0.08 and 0.15 kg) $\times$ three friction coefficients (0.5, 1.0, 2.0) $\times$ six seeded geometric scenes (54 trials, seed 2026). The first version succeeded in 44 of 54 trials (81.5%), and every failure came from the low-friction column (Table 16, Figure 11): at friction 0.5 only 8 of 18 trials succeeded (0% for the 0.04 kg cube, 33% for 0.08 kg and 100% for 0.15 kg), whereas all 36 trials at friction 1.0 and 2.0 succeeded. By mass, the initial controller scored 12/18 (66.7%) at 0.04 kg, 14/18 (77.8%) at 0.08 kg and 18/18 at 0.15 kg. Low friction is therefore the decisive factor, and the pattern across masses does not simply track weight (the lightest cube failed every time, the heaviest never did); the archive does not diagnose the cause. After the controller was adjusted, it succeeded in 54 of 54 trials on the development seed (2026) and in 54 of 54 on a separate hold-out seed (2027), with no ground contact in either set (the archive records the outcome but not a line-by-line description of the adjustment). The grasp phase of one scripted episode is shown in Figure 12.

Table 16. Scripted baseline: success over the mass $\times$ friction grid (54 trials per run).

Run	Overall	Friction 0.5	Friction 1.0	Friction 2.0
Initial controller (seed 2026)	44/54 (81.5%)	8/18 (44.4%)	18/18	18/18
Adjusted controller, development (seed 2026)	54/54 (100%)	18/18	18/18	18/18
Adjusted controller, hold-out (seed 2027)	54/54 (100%)	18/18	18/18	18/18

Cubes of 5 cm, six seeded geometric scenes, no obstacles. Wilson 95% interval for 54/54: 93.4–100.0%.

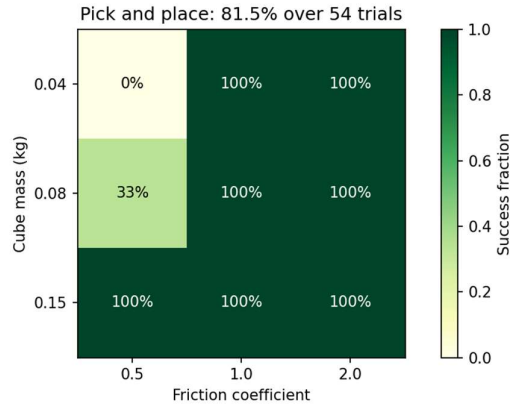


Figure 11. Initial scripted-controller success over the mass $\times$ friction grid (54 trials). All failures occur at friction 0.5.

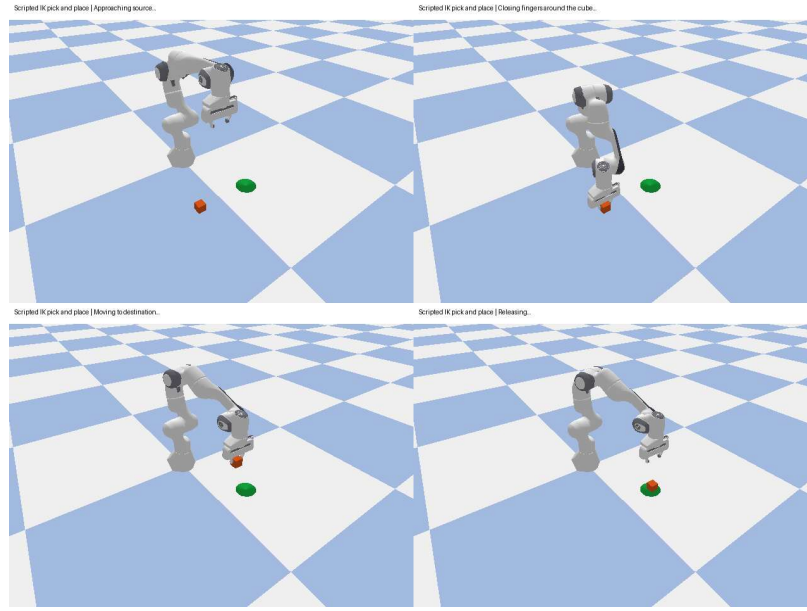


Figure 12. Four frames of a scripted pick-and-place episode: approach, closing the fingers around the cube, moving to the destination, and releasing onto the green marker (frames from the project's recorded GIF).

3.6 Learned Panda controllers: from the nominal task to scene generalisation

3.6.1 Nominal task: lifting and pick-and-place

With the redesigned Cartesian interface, a behaviour-cloning and DAgger policy followed by only 1,024 PPO steps solved the nominal task (fixed 5 cm cube, 0.08 kg, friction 2.0) on randomised start and goal positions. Twenty teacher demonstrations (all successful) seeded the training. The development validation success of the policy over the imitation rounds was 0%, 0% and 90% (Figure 13), so two rounds of DAgger were needed before the policy could complete the task, and the loss alone was a poor guide to quality: the action-fitting loss changed little (0.0085, 0.0096, 0.0058) while the success went from none to nine in ten. The evaluation of the final nominal checkpoint on 50 randomised hold-out scenes (seeds 5000–5049) gave 50/50 successes with a mean placement error of 0.65 cm (Table 17). The grasp-and-lift-only policies gave 50/50 on both the near-cube evaluation (seeds 3000–3049, a separate checkpoint) and the full-start evaluation (seeds 4000–4049); the “placement error” reported for these (about 34 cm) is not meaningful because they were not asked to place. The Wilson lower bound for 50/50 is 92.9%.

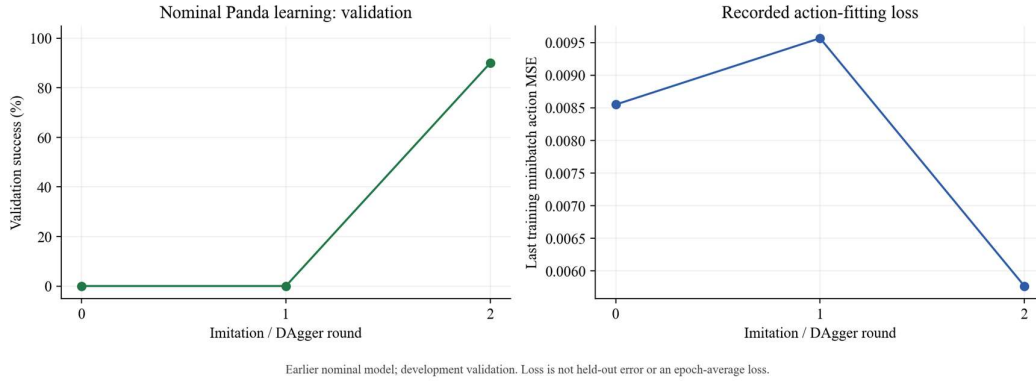


Figure 13. Nominal Panda learning: development validation success (left) and the recorded action-fitting loss (right) over the imitation and DAgger rounds. The loss is the last training minibatch value, not a held-out error.

Table 17. Nominal Panda checkpoints (fixed 5 cm cube; behaviour cloning + DAgger + 1,024 PPO steps).

Task and evaluation	Seeds	Episodes	Successes	Mean error	95% Wilson interval
Grasp and lift, near-cube start (separate checkpoint)	3000–3049	50	50 (100%)	n/a	92.9–100.0%
Grasp and lift, full start	4000–4049	50	50 (100%)	n/a	92.9–100.0%
Full pick-and-place, hold-out	5000–5049	50	50 (100%)	0.65 cm	92.9–100.0%

3.6.2 Physics randomisation, obstacles and the quarter-turn gripper

Table 18 shows how the policies fared in the stress suites as the project progressed. The nominal policy, which had never seen other objects, still succeeded in 23 of 30 episodes of the original ten-case suite on its development seed and 21 of 30 on a second seed, but the failures were systematic: small and large objects (0/3 each on the second seed) and the 24 cm blocking obstacle (0/3, with obstacle contact in every episode). Training on domain-randomised demonstrations (mass, friction, size and shape) raised the second-seed result to 26 of 30, with small and cylinder objects now handled, although the large object (1/3) and blocking obstacle (1/3, with contact in all three) remained weak. Of the 26 successes, 25 were free of obstacle contact.

Two further changes addressed the obstacle. The obstacle-aware profile (32 observation values with obstacle geometry) reached 31 of 36 collision-free successes on its hold-out suite of twelve cases (seed 11,000). Its remaining failures involved small and large objects and the three obstacle cases (blocking, large-blocking and cylinder-blocking), each with an obstacle-contact episode. The quarter-turn profile (a 90° gripper rotation with a teacher that releases higher and opens before withdrawing) removed most of these: 20 of 20 regression episodes, 50 of 50 hold-out episodes (6 cm box, 24 cm obstacle, seeds 24,000–24,049) with a mean error of 0.91 cm and no obstacle contact, and 34 of 36 on the stress suite. Both stress failures were again associated with the geometry: the low obstacle (2/3; mean placement error for that case 7.5 cm) and the cylinder with a blocking obstacle (2/3, with one obstacle-contact episode).

Table 18. Stress-suite results as the Panda controller gained capabilities.

Checkpoint / profile	Seed	Suite	Successes	Main failures
Nominal (nominal profile), development	6,000	10 cases × 3	23/30	small 1/3 (mean error 21.7 cm); heavy 2/3; large 2/3; blocking obstacle 0/3 (3 contacts)
Nominal, second seed	8,000	10 × 3	21/30	small 0/3; large 0/3; blocking obstacle 0/3 (3 contacts)
Domain-randomised BC + DAgger + PPO	8,000	10 × 3	26/30 (25 collision-free)	large 1/3; blocking obstacle 1/3 (3 contacts)
Obstacle-aware v1, development	9,000	10 × 3	25/30	large 1/3; low obstacle 2/3; blocking obstacle 2/3; heavy-slippery 2/3
Obstacle-aware v1, hold-out	11,000	12 × 3	31/36 (31 collision-free)	small 2/3; large 2/3; blocking 2/3; large-blocking 2/3; cylinder-blocking 2/3 (contacts in 3 cases)
Quarter-turn v2, regression	2,000–2,019	20 episodes	20/20	none (6 cm box, 24 cm obstacle)
Quarter-turn v2, hold-out	24,000–24,049	50 episodes	50/50	none; mean error 0.91 cm
Quarter-turn v2, stress	25,000	12 × 3	34/36	low obstacle 2/3; cylinder-blocking 2/3 (1 contact)

Suites are finite and the cases are not independent random samples. “Collision-free” excludes episodes in which the arm or object touched the obstacle.

3.6.3 Varied scenes, wide positions and rotated boxes

The 17-case suite (Table 10) was introduced to test sources and destinations across a wider workspace and boxes rotated by 45° (Table 19). The obstacle-v2 checkpoint scored 26 of 34. It failed both wide-position cases (0/2 each, with mean errors of 47 cm and 44 cm), the rotated large box (0/2) and the cylinder behind a blocking obstacle (0/2, both with obstacle contact). A varied-scene refinement trained on 84,966 aggregated demonstrations from eight conditions raised the hold-out result on the same paired seeds (32000–32001) to 32 of 34. The only remaining failure was the 45°-rotated 6 cm box (0/2), which the project record explicitly flags as unsolved: no universal-orientation claim was made at this stage.

The next step added gripper yaw alignment (the yaw-aligned profile), which uses the simulator’s known initial object yaw. The first yaw-aligned candidate produced a result that needs careful interpretation. On its *development* seeds (33,000–33,001) it succeeded in only 8 of 34 episodes, yet the same checkpoint (identical SHA-256) succeeded in 31 of 34 episodes on the hold-out seeds (34,000–34,001), failing only the rotated boxes (rotated small box 1/2, rotated large box 0/2). The project archive contains no explanation for this 23-episode difference between two sets of seeds for one checkpoint. The failed development episodes ended with mean placement errors of 17–47 cm, so the object was never carried to the goal. Possible explanations, none of which was tested, include a sensitivity of the yaw-aligned profile to the sampled layouts (for example the wrist-joint limit that constrains the chosen gripper yaw) or an evaluation artefact, and with only two seeds per case the suite cannot separate them. We report both numbers because quoting only the favourable one would be misleading (see Section 4.3).

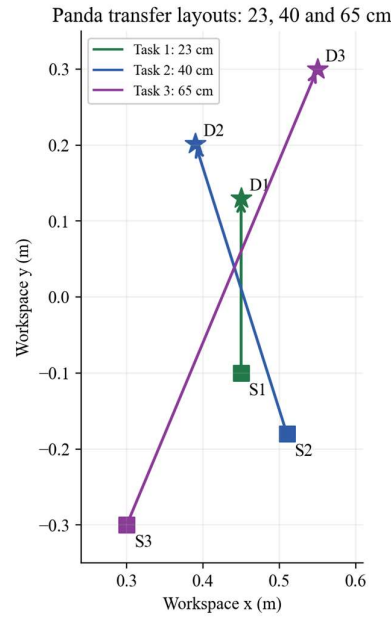
Table 19. Scene generalisation on the 17-case suite (two paired seeds per case, 34 episodes).

Checkpoint (profile)	Seeds	Result	Wilson 95%	Failed cases
Obstacle v2 (quarter_turn_v2) – baseline	32,000–32,001	26/34	60.0–87.6%	cylinder-blocking 0/2 (contact); wide left 0/2; wide right 0/2; rotated large box 0/2
Varied-scene BC (quarter_turn_v2)	32,000–32,001	32/34	80.9–98.4%	rotated large box 0/2
Yaw-aligned candidate (yaw_aligned_v3), development	33,000–33,001	8/34	12.4–40.0%	most cases failed; successes in reference 1/2, slippery 1/2, large 1/2, heavy-slippery 1/2, wide right 2/2, rotated large box 2/2
Yaw-aligned candidate, hold-out (same SHA-256)	34,000–34,001	31/34	77.0–97.0%	rotated small box 1/2; rotated large box 0/2

The second and third rows use the same pairs of seeds; the fourth and fifth rows are one checkpoint on two different sets of seeds.

3.7 Random diagonal layouts, 23/40/65 cm transfers and the persistent scene

Three further capabilities were added by focused behaviour cloning (Table 20). **Diagonal layouts.** A policy refined on 25 ordinary and 25 random diagonal demonstrations (25,460 aggregated samples; one round) succeeded in 20 of 20 fresh diagonal layouts (seeds 42,000–42,019) with a mean placement error of 1.12 cm. A three-episode replay of seed 622,849, which exposed a failure of the preceding checkpoint (2/3, mean error 14.6 cm), was solved 3/3 (0.87 cm). Because that replay seed was used to diagnose the earlier failure, it is evidence that the specific failure was repaired, not an independent test. **Distance tasks.** Refinement on the three fixed 23, 40 and 65 cm layouts plus random diagonals (20,720 samples) gave 20 of 20 on fresh diagonals (seeds 45,000–45,019; 1.03 cm) and 3/3 on the three fixed tasks, compared with 2/3 for the previous checkpoint, although those exact layouts were part of the training demonstrations (Figure 14). **Persistent scene.** In the persistent scene (three cubes present from the start, arm and world never reset) the persistent-scene behaviour-cloning policy completed all three tasks (seed 48,000) with errors of 1.99, 2.16 and 1.40 cm in 226, 202 and 301 steps, and the final report confirmed that all three objects were settled.



Squares: sources; stars: destinations. Arrows show straight-line separation, not measured arm trajectories.

Figure 14. Source (square) and destination (star) positions of the three fixed transfer tasks of 23, 40 and 65 cm; arrows are straight lines, not measured trajectories.

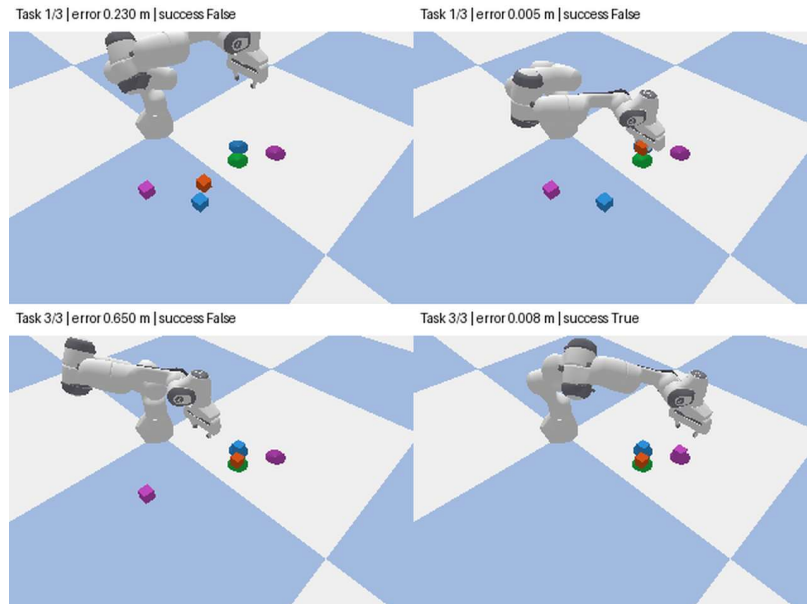


Figure 15. Frames from the recording of the persistent three-object scene (tasks 1 and 3, before and after placement). The overlay shows the recorder's running status; success is only set after ten settled steps, so it lags the placement error. The source GIF is low resolution.

Table 20. Layout, distance and persistent-scene results for the specialist checkpoints.

Checkpoint and test	Seeds	Layouts	Successes	Mean error	Note
Varied-scene BC, diagonal replay	622,849	3 diagonal	2/3	14.6 cm	Diagnosed failure
Diagonal policy, replay	622,849	3 diagonal	3/3	0.87 cm	Same seed as the diagnosis
Diagonal policy, hold-out	42,000–42,019	20 random diagonals	20/20	1.12 cm	Wilson 83.9–100.0%
Diagonal policy, 23/40/65 cm tasks	2,000	3 fixed	2/3	21.9 cm	Before distance refinement
Distance policy, 23/40/65 cm tasks	2,000	3 fixed	3/3	1.69 cm	Layouts in training data
Distance policy, hold-out	45,000–45,019	20 random diagonals	20/20	1.03 cm	Wilson 83.9–100.0%
Persistent policy, three-object scene	48,000	3 fixed, persistent	3/3	1.99 / 2.16 / 1.40 cm	All settled; fixed layouts

3.8 The final shared Panda checkpoint

To avoid keeping a separate specialist for each capability, the project built one shared checkpoint by balanced behaviour cloning over the four saved datasets (6,000 samples from each) and then added yaw, size and obstacle demonstrations: 10 conditions × 5 teacher rollouts, all 50 successful, final dataset 51,682 samples, 100 epochs and no PPO. It passed every check reported in the documentation (Table 21): the three 23/40/65 cm tasks (3/3, mean error 1.45 cm), the persistent three-object sequence (3/3, errors 0.54, 0.09 and 0.79 cm in 228, 199 and 260 steps, all objects settled), 20 of 20 fresh random diagonal layouts (seeds 56,000–56,019, mean error 1.00 cm, Wilson interval 83.9–100.0%), and 34 of 34 episodes in the 17-case suite (interval 89.8–100.0%, mean per-case placement error 1.13 cm, worst case 2.07 cm). Figure 16 compares six Panda checkpoints, with totals ranging from 26 of 34 to 34 of 34 (the PPO column is a side branch from the intermediate imitation checkpoint), and Figure 17 shows the case-by-case outcome and error of the final checkpoint. Figure 18 shows the placement errors of the 20 diagonal episodes, all of which sit well inside the 5 cm tolerance.

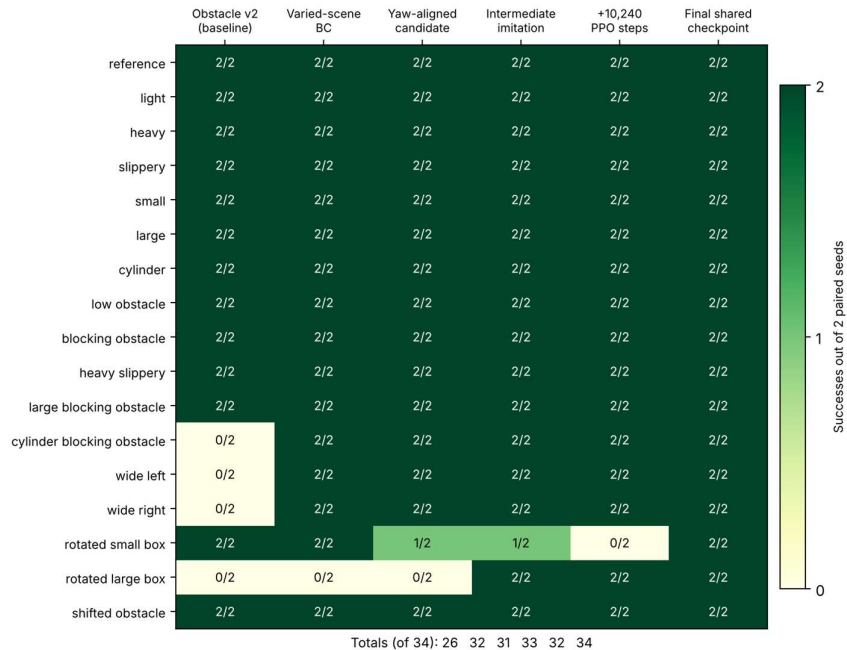


Figure 16. Successes out of two paired seeds for each of the 17 stress cases across six Panda checkpoints (columns; the PPO column branches from the intermediate imitation checkpoint), with totals below. The yaw-aligned candidate column uses its hold-out result (31/34); its development result was 8/34 (Section 3.6.3).

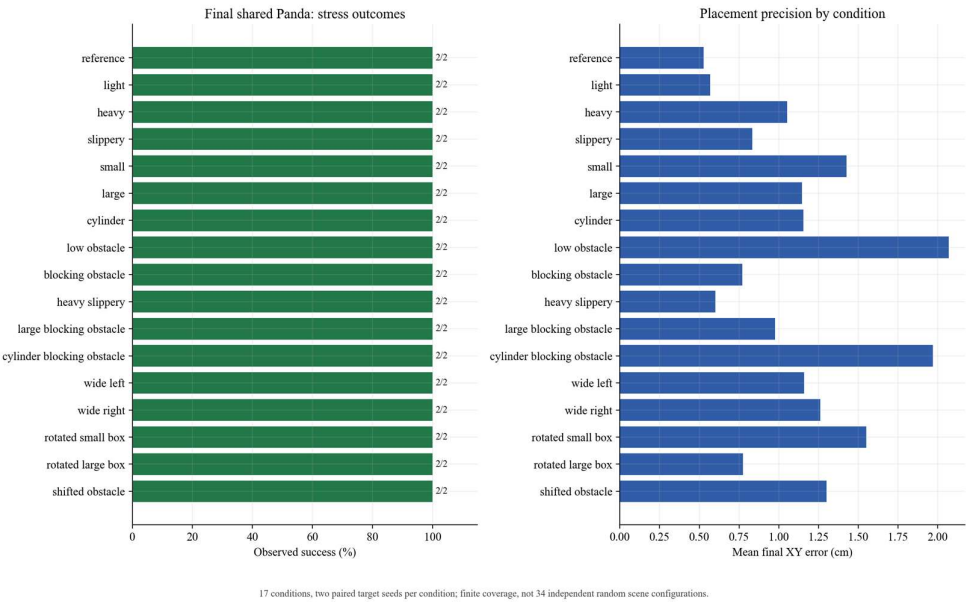


Figure 17. Final shared checkpoint: success (left) and mean placement error (right) in each of the 17 stress cases (two paired seeds per case).

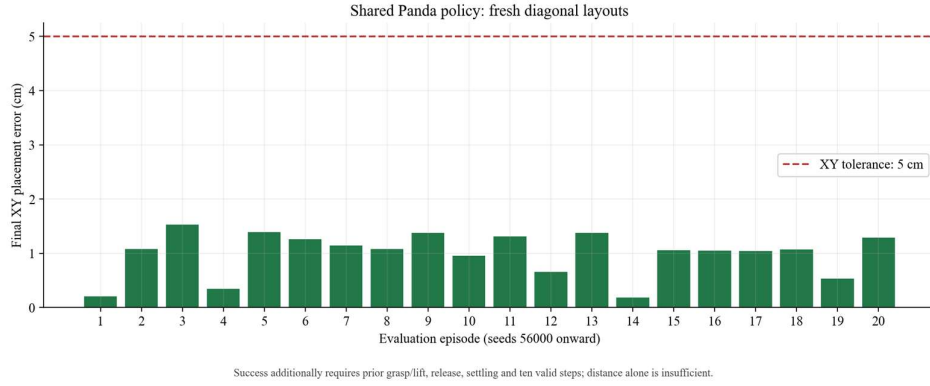


Figure 18. Final XY placement error of the 20 fresh random diagonal episodes (seeds from 56,000). The dashed line is the 5 cm tolerance. Success also requires prior grasp and lift, release, settling and ten valid steps.

Table 21. Final shared checkpoint (panda_simulation_policy.zip, yaw_aligned_v3 profile).

Check	Seeds	Episodes	Successes	Mean error	Scope / overlap with training
23 / 40 / 65 cm tasks	2,000	3 (fixed layouts)	3/3	1.45 cm	Exact layouts are in the demonstrations
Persistent three-object sequence	56,000	3 tasks, one scene	3/3	0.54 / 0.09 / 0.79 cm	Same fixed layouts; all three settled
Fresh random diagonal layouts	56,000–56,019	20	20/20	1.00 cm	New seeds; same layout generator
17-case stress suite	56,000–56,001	34 (17 × 2)	34/34	≈ 1.13 cm (case mean)	Yaw cases overlap the yaw-refinement demonstrations

Wilson 95% intervals: 3/3 43.9–100.0%; 20/20 83.9–100.0%; 34/34 89.8–100.0%.

How strong is the 34/34 result?

It is the strongest result in the project, but it measures the checkpoint on conditions that overlap those it was trained on: the 4/5/6 cm, 45° box and tall-obstacle conditions of the stress suite are near the yaw-refinement demonstrations (box sizes 4 and 6 cm at yaw 0.4, 0.785, 1.2 and -0.785 rad; 6 cm box and a cylinder behind a 24 cm obstacle).

Only two paired seeds were used per case, so a single failure in 34 would have changed the headline.

The 20/20 diagonal result uses fresh seeds but the same layout generator as the training data, and the 3/3 task and persistent results use exactly the layouts that were demonstrated.

The yaw cases receive the true yaw from the simulator through the low-level controller; the neural policy is not asked to perceive it.

3.9 PPO continuation of an intermediate shared checkpoint

The last experiment asked whether a short PPO continuation (RQ4, second part) improves an imitation-trained checkpoint (Table 22, Figure 19). Starting from the intermediate yaw-refined shared candidate (33/34 on the stress suite and 20/20 on diagonals), PPO was resumed for 10,240 additional steps (ten rollouts of 1,024 steps) with learning rate 10^{-5} , target KL 0.001 and a single seed, in the obstacle-aware random-layout setting. The training log (Figure 20) is flat: validation success was 100% at 0, 10,000 and 10,240 steps, held height 0.18 m throughout, the mean evaluation reward moved only from 89.24 to 88.78, the clip fraction was

0 and the approximate KL stayed at $5\text{--}7 \times 10^{-4}$. The policy standard deviation stayed near 0.082. The update was therefore small, as intended.

On identical test seeds (54,000 onwards) the continuation was not better and was slightly worse. Stress success dropped from 33/34 to 32/34 (the extra failure was the 4 cm box rotated by 45° , 0/2 against 1/2 before), the mean diagonal placement error rose from 11.1 mm to 18.5 mm although success stayed 20/20, and the persistent sequence still passed 3/3 but with errors near 1.9 cm in every task instead of the smaller errors of the separately refined checkpoint (0.1–0.8 cm). The recommendation recorded with the experiment is to reject the PPO continuation as an improvement and to prefer a separately refined imitation checkpoint, which was then tested at new seeds (56,000–56,019) and became the final shared model. The experiment used one PPO seed and a short budget, so it supports no general claim about PPO, only the statement that this short continuation did not help.

Table 22. Paired comparison: intermediate imitation checkpoint against its 10,240-step PPO continuation (seed 54,000 onwards).

Measure	Before PPO	After 10,240 PPO steps	Final shared checkpoint (separate; seed 56,000)
17-case stress suite (34 episodes)	33/34	32/34	34/34
Diagonal layouts (20 episodes), success	20/20	20/20	20/20
Diagonal layouts, mean placement error	1.11 cm	1.85 cm	1.00 cm
23/40/65 cm tasks	not run	3/3 (1.86 cm)	3/3 (1.45 cm)
Persistent three-object sequence	not run	3/3 (1.93–1.95 cm per task)	3/3 (0.09–0.79 cm per task)
Failed stress case	rotated small box 1/2	rotated small box 0/2	none
Training validation success	100%	100% (flat)	—

Single PPO seed, learning rate $1e-5$, target KL 0.001. The third column is a separately refined imitation checkpoint, not a PPO result.

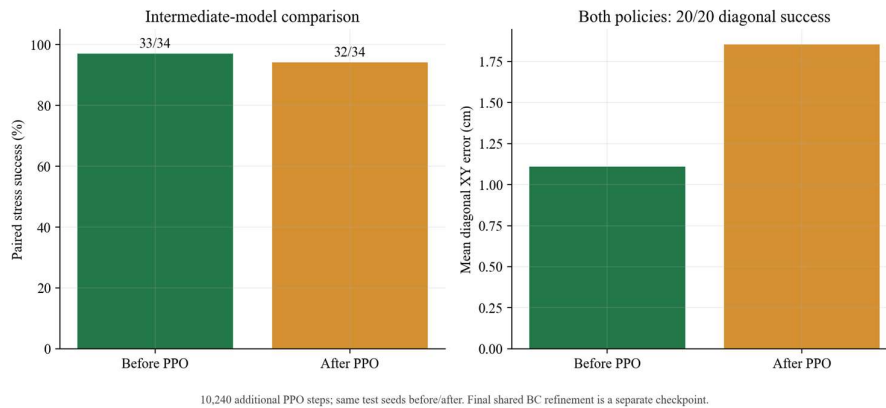


Figure 19. Paired comparison before and after 10,240 PPO steps: stress success (left) and mean diagonal XY error (right).

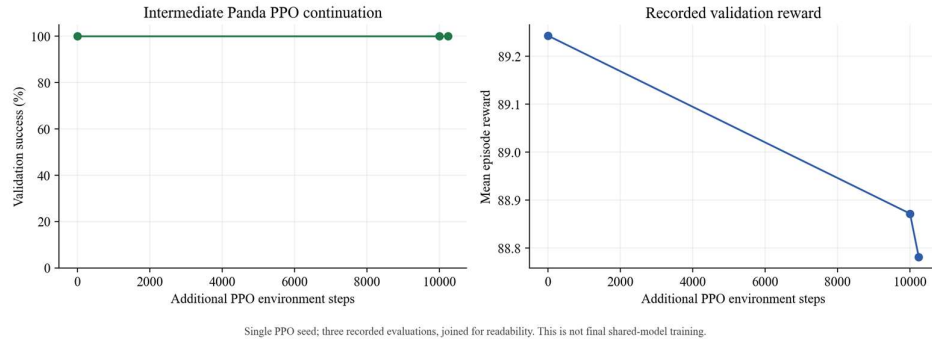


Figure 20. Validation success and mean reward during the 10,240-step PPO continuation (three recorded evaluations, joined for readability; single seed).

3.10 Overall summary of results

Figure 21 places the main results on one scale with Wilson intervals, and Table 23 lists every checkpoint with its method, its main evaluation and a truncated SHA-256 (full hashes in Appendix A). The two KUKA PPO checkpoints sit at the left of the figure, below the analytic IK baseline, whereas the Panda checkpoints (imitation-trained) sit near 100%. The two groups should not be compared directly: the KUKA task uses 500 fresh random targets and a PPO policy trained by trial and error, while the Panda tasks use much smaller finite test sets and policies trained from teacher demonstrations. The figure's message is the contrast in the width of the intervals: 296/500 is known to ± 4 percentage points, whereas 3/3 is compatible with a true rate as low as 44%.

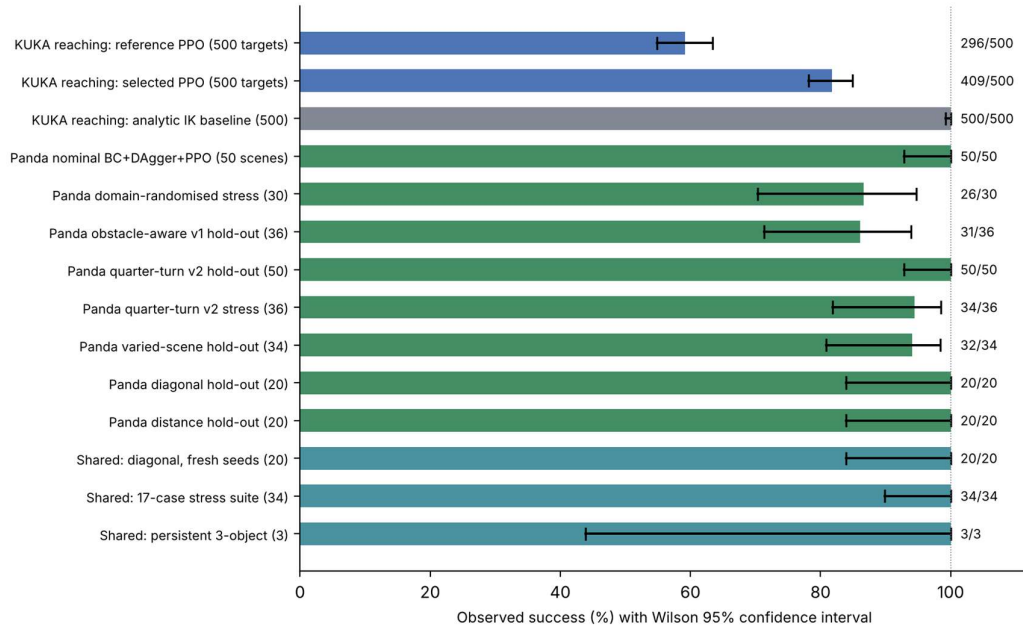


Figure 21. Success rates of the main evaluations with 95% Wilson intervals (counts on the right). KUKA bars are PPO checkpoints or the IK baseline on 500 targets; Panda bars are imitation-trained policies on finite test sets of 3 to 50 episodes.

Table 23. Checkpoint inventory (SHA-256 truncated to 12 characters).

Checkpoint	Method	Principal evaluation	Result	SHA-256
reference_policy	PPO, early KUKA run (starting point)	500 targets, seeds 2000–2499	296/500	0081c3ac4a59
reaching_policy	PPO, legacy-reward continuation (seed 19)	500 targets, seeds 2000–2499	409/500	ab37880c24be
panda_pick_place_policy	BC + DAgger + 1,024 PPO steps (nominal)	50 hold-out scenes	50/50	c7881804ad4a
panda_lift_policy	BC + DAgger + 1,024 PPO steps (lift)	Full-start lift, 50 trials	50/50	f11724fe40c8
panda_randomized_policy	Domain-randomised BC + DAgger + PPO	10-case stress, 30 episodes	26/30	a16a94daf888
panda_obstacle_policy	BC + DAgger (obstacle-aware v1)	12-case hold-out, 36 episodes	31/36	a0955be9aaa1
panda_obstacle_policy_v2	BC + DAgger (quarter-turn v2)	Hold-out 50; stress 36	50/50; 34/36	9315bde0d67d
panda_varied_scene_policy	BC (varied scenes)	17-case suite, 34 episodes	32/34	05a84888d881
panda_diagonal_policy	Focused BC (diagonals)	20 fresh diagonals	20/20	d34ac53f7dbd
panda_distance_policy	Focused BC (23/40/65 cm)	20 fresh diagonals; 3 tasks	20/20; 3/3	e1345410bc9f
panda_persistent_policy	BC (persistent scene)	Three-object sequence	3/3	0231d8345bd1
panda_simulation_policy	Balanced BC + yaw refinement (shared)	Diagonals 20; stress 34; tasks 3; persistent 3	20/20; 34/34; 3/3; 3/3	aba0830b49e8

4. Discussion

4.1 Interpretation of the main findings

The four research questions can now be answered, each with its own level of confidence.

RQ1 (relative-vector observation). Probably helpful, not demonstrated. The three-seed mean final evaluation success rose from 34.4% to 60.0% and the best in-training evaluation from 67.8% to 83.3%, but seeds ranged from 16.7% to 90.0% and the difference is far from statistically significant. The observation change is cheap and well motivated, so it remains a sensible default, but a claim that it “improves” PPO reaching needs more seeds (Henderson et al., 2018; Agarwal et al., 2021). The final reaching checkpoint also belongs to the basic-observation lineage, so the ablation result was not carried into the deliverable.

RQ2 (dense progress reward). Not supported. In an equal-budget, paired-seed continuation the shaped reward lost to the original distance reward on all three seeds, by 8.5 to 20.5 percentage points on the hold-out targets. A near-potential-based bonus did not give the guarantees of strict potential-based shaping (Ng, Harada and Russell, 1999), because the discount factor, action penalty and ground-contact penalty were additional terms, and the simplest reading is that the extra terms changed the optimisation landscape without improving the reachable optimum. The experiment was a continuation of an existing checkpoint, not training from scratch, so it does not exclude shaping helping early learning.

RQ3 (PPO from scratch on Panda). Answered negatively. Two 1,000,000-step runs produced 0% success, and their failure modes (learning to grasp but dropping, or never grasping) point at exploration and reward-structure problems rather than at insufficient steps. What worked was changing the problem: a Cartesian action interface anchored to the measured pose, IK-generated joint commands, physically grounded success criteria and demonstrations.

RQ4 (generalisation of demonstration-trained policies; value of short PPO). The final shared checkpoint passed every saved test, including 34/34 on the 17-case suite, but the evidence is finite and overlaps with training conditions (Section 3.8). The 10,240-step PPO continuation did not improve a policy that already passed validation at 100%, which is unsurprising: with 100% success there is little advantage signal, and the learning rate (10^{-5}), target KL (0.001) and small policy standard deviation (about 0.08) were chosen to keep the update small, so the update could only add noise to a policy that was already near its ceiling.

The most important finding is a contrast. On the KUKA task, PPO with a joint-delta action interface learned a useful reaching skill in about 500,000 steps, but still failed on 18% of unseen targets, behaved as a saturated bang-bang controller, and was beaten by a classical IK baseline that succeeded on every target. On the Panda task, the same algorithm in a joint-space interface learned nothing in twice that budget. This pattern is typical of what the literature reports for robotic RL: success depends on problem formulation (state, action space, reward, resets) at least as much as on the algorithm (Kober, Bagnell and Peters, 2013; Ibarz et al., 2021).

4.2 Why the redesigned Panda problem became tractable

Several design decisions were made together, so the project cannot say how much each contributed (no ablation of the redesign was run). It is nevertheless possible to say what each decision does mechanically.

- **Cartesian increments anchored to the measured pose.** A policy that chooses 1 cm steps of the end effector has a much more direct relationship between action and task progress than one that chooses seven joint deltas, and anchoring each step to the *measured* position stops commands from accumulating behind a stalled arm.
- **Inverse kinematics with a fixed gripper orientation.** IK removes the need to learn the arm's kinematics and keeps the gripper pointing down (or at the commanded yaw). This is a strong prior: the policy decides where to move, not how.
- **Rewards that pay achievements only once.** Reward version 3 pays new bests and first achievements, so oscillating contact or height cannot farm reward, and the success definition requires a physically meaningful sequence. This addresses the early reward flaw in which dropping early was cheaper than carrying.
- **A hand-written feedback teacher.** Demonstrations bypass the exploration problem that defeated PPO. The teacher is cheap and consistent, and DAgger (Ross, Gordon and Bagnell, 2011) corrects the distribution shift of behaviour cloning by labelling the states the learner actually visits. In the nominal run two rounds were needed: validation success was 0%, 0% and then 90%, while the loss barely changed.
- **Focused refinement and balanced data.** Each new capability (obstacle, quarter-turn gripper, varied scenes, diagonals, distances, persistence, yaw) was added by collecting demonstrations for the new condition and refitting, with seeds partitioned in advance and the new checkpoint compared with the old on the same seeds. The final shared policy sampled evenly from four saved datasets to avoid forgetting.

These choices follow a well-established pattern: demonstrations and structured action spaces are standard ways of making hard manipulation problems tractable (Argall et al., 2009; Nair et al., 2018; Rajeswaran et al., 2018; Zhu et al., 2018), and the cost is that the resulting policy is only as capable as its teacher and its demonstration distribution. That is why the report calls the final policies imitation-trained and does not credit them to reinforcement learning.

4.3 Limitations and threats to validity

4.3.1 Internal validity: statistics and selection

- **Small and finite test sets.** The stress suite uses two paired seeds per case, and the persistent scene uses three tasks. A perfect score on 34 episodes is compatible with a true success rate as low as 89.8%, and 3/3 with as low as 43.9% (Wilson, 1927). The suites are probes and not random samples of scenes.
- **Unexplained development/hold-out gap.** One checkpoint (identical SHA-256) scored 8/34 on development seeds and 31/34 on hold-out seeds. Until this is understood, any single small suite should be regarded as noisy, and the later 34/34 should be interpreted with that noise in mind.
- **Overlap between training and test conditions.** The fixed 23/40/65 cm tasks and the persistent scene are part of the training demonstrations, and the yaw and size conditions of the stress suite are close to the yaw-refinement demonstrations. These

tests show that the policy retains the demonstrated skills, not that it generalises beyond them.

- **Selection effects in KUKA training.** The “best” checkpoint during training was chosen on a small fixed set that was also used to report progress, so in-training success of 99–100% is optimistic; the same lineage scored 59.2% on 500 fresh targets. Only the final 500-target evaluation is free of this bias.
- **Few seeds.** The observation ablation and avoidance runs use three seeds; the SAC, precision and two-sphere runs use one; and the PPO continuation uses one. Spread across seeds was as large as the effects being measured.
- **Multiple comparisons and researcher degrees of freedom.** Several reward, observation and algorithm variants were tried, and the selected checkpoint is the best of six candidates. The use of validation seeds separate from the final seeds mitigates but does not remove this.
- **No saved robustness-to-noise results.** A noise script exists, but its outputs are not in the archives, so no claim about sensor-noise robustness is made.

4.3.2 Construct validity: what “success” measures

- **Privileged information.** All Panda policies observe exact object, goal and obstacle state. The final policy also relies on the simulator-provided initial object yaw to orient the gripper, supplied through the low-level controller. The policy does not perceive orientation.
- **Teacher dependence.** The demonstration teacher has hand-set thresholds (approach height, transport height of 21 or 34 cm, release clearances). The learned policy inherits its strategy and its blind spots, such as the unsolved rotated-box conditions before yaw alignment.
- **Collision model.** The KUKA obstacle experiments tested only the end-effector point against each sphere, so the arm’s links could pass through an obstacle without penalty. The Panda obstacle environment checks contact for the arm and object, but with one known fixed box.
- **Success thresholds.** A 5 cm placement tolerance is generous for precision assembly (though actual errors were around 1 cm), and the 5 cm reaching threshold is not a measure of precise positioning (the 2 cm variant was unstable).

4.3.3 External validity

- **Simulation only.** No hardware experiments were run. PyBullet’s contact, friction and actuator models are approximations, and sim-to-real transfer typically requires dynamics randomisation, calibration and robust perception (Tobin et al., 2017; Peng et al., 2018; Zhao, Queralta and Westerlund, 2020).
- **Narrow object and scene family.** Objects are upright boxes or cylinders of 4–6 cm, there is at most one known box obstacle, and positions lie in a bounded region. Tilted objects, clutter, moving obstacles and arbitrary shapes are outside the supported contract.
- **No learned perception.** The simulated camera only records media. End-to-end visuomotor control (Levine et al., 2016; Kalashnikov et al., 2018) is not attempted.
- **Single platform.** Everything ran on one software stack with fixed versions on CPU, which helps reproducibility but means nothing is known about other physics versions.

4.4 Implications and recommendations

1. **Design the interface before scaling the training.** Changing the action space and the reward turned an unlearnable problem into one solved with a few thousand demonstrations; adding more PPO steps to the failed design would have been the expensive option.
2. **Use demonstrations when a hand-written controller exists.** A simple feedback teacher plus DAgger gave reliable policies quickly and provided a natural target to compare against. Report clearly that the result is imitation, not RL.
3. **Keep a classical baseline.** The IK controller reached every target and outperformed learned reaching, which puts the learned policy's value into perspective. Learning's appeal lies in flexibility, not in beating a well-posed analytic solution.
4. **Separate development from evaluation seeds, hash checkpoints and refuse to overwrite results.** These practices made it possible to detect the anomaly in Section 3.6.3 and to reject the PPO continuation honestly. A negative result was preserved and reported instead of being hidden.
5. **Use more seeds and report intervals.** With three seeds and standard deviations as large as the effects, any ranking of methods is tentative (Agarwal et al., 2021).
6. **State the scope in the same sentence as the result.** "34/34" without "17 cases, 2 seeds, overlapping with demonstrations, exact state" is easy to mistake for general reliability.

4.5 Future work

- **RL that works on the redesigned interface.** Try SAC, which learned fastest on the KUKA obstacle task (Haarnoja et al., 2018), in the Cartesian Panda environment, with hindsight experience replay for the sparse success signal (Andrychowicz et al., 2017) or demonstration-augmented updates (Nair et al., 2018; Rajeswaran et al., 2018). Run an ablation that removes one element of the redesign at a time.
- **Larger PPO fine-tuning with proper seeds.** The negative PPO result is for one seed and 10,240 steps. A fair test needs several seeds, a lower-performing starting checkpoint with headroom and independent test seeds.
- **Curriculum and randomisation.** Combine the stages (reach, lift, place) with systematic randomisation of friction, mass, controller gains and latency, then evaluate on a held-out distribution rather than on repeated seeds.
- **Perception.** Replace exact state with camera images or estimated poses (Levine et al., 2016), including an orientation estimate so that the policy, not the simulator, determines the grasp yaw.
- **Broader scenes.** Multiple and moving obstacles, clutter, tilted objects and arbitrary shapes, together with whole-arm collision checking on KUKA.
- **Statistics and documentation.** Resolve the 8/34 versus 31/34 discrepancy by re-running the same checkpoint on many seeds; run and save the noise-robustness script; and extend the suite to more seeds per case so that intervals narrow.
- **Hardware.** Only after the above, test on a physical arm with safety limits, force control and an independent evaluation protocol.

5. Conclusion

This project built a seeded, tested and reproducible PyBullet codebase for two robot arms and used it to study how well reinforcement learning and demonstration-assisted learning can control them. On the KUKA LBR iiwa, PPO learned to reach random targets within 5 cm in 81.8% of 500 unseen episodes (95% interval 78.2–84.9%), an improvement over the 59.2% of its starting checkpoint but below the 100% of an analytic IK baseline. A relative-vector observation appeared to help (34.4% to 60.0% mean final evaluation success over three seeds), but seed variance was too large for a firm conclusion, and a dense shaped reward did not improve on the original reward in any of three paired seeds. Obstacle avoidance with one sphere was learned reliably in training (87.8% mean final evaluation success), with SAC reaching 100% in a single run.

On the Franka Panda, PPO from scratch with joint-space actions achieved 0% success in two 1,000,000-step runs. Redesigning the interface (Cartesian increments, inverse kinematics, a physical gripper, success defined by grasp, lift, release and settling) and learning from a training-time teacher by behaviour cloning and DAgger produced policies that passed every saved test: the final shared checkpoint scored 34/34 on a 17-case stress suite, 20/20 on fresh diagonal layouts, 3/3 on 23/40/65 cm transfers and 3/3 on a persistent three-object scene, with typical placement errors near 1 cm. A short PPO continuation did not improve an earlier imitation checkpoint (stress 33/34 to 32/34; mean diagonal error 11.1 mm to 18.5 mm).

The conclusion drawn from the evidence is deliberately modest. The results hold in simulation, on finite and sometimes overlapping test sets, with exact state information and simulator-provided object yaw, and they include an unexplained difference between development and hold-out performance of one checkpoint. They show that careful problem formulation, demonstrations and disciplined evaluation can make hard manipulation tasks tractable, and they document where plain PPO fails. They do not show general reliability, camera-based perception or real-world transfer. These are the natural next steps.

References

References follow the Birmingham City University Harvard style (Cite Them Right). Web and preprint sources were accessed on 6 October 2026.

- Agarwal, R., Schwarzer, M., Castro, P.S., Courville, A. and Bellemare, M.G. (2021) 'Deep reinforcement learning at the edge of the statistical precipice', *Advances in Neural Information Processing Systems*, 34, pp. 29304–29320.
- Andrychowicz, M., Wolski, F., Ray, A., Schneider, J., Fong, R., Welinder, P., McGrew, B., Tobin, J., Abbeel, P. and Zaremba, W. (2017) 'Hindsight experience replay', *Advances in Neural Information Processing Systems*, 30, pp. 5048–5058.
- Argall, B.D., Chernova, S., Veloso, M. and Browning, B. (2009) 'A survey of robot learning from demonstration', *Robotics and Autonomous Systems*, 57(5), pp. 469–483.
- Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J. and Zaremba, W. (2016) *OpenAI Gym*. arXiv:1606.01540. Available at: <https://arxiv.org/abs/1606.01540> (Accessed: 6 October 2026).
- Coumans, E. and Bai, Y. (2016–2021) *PyBullet, a Python module for physics simulation for games, robotics and machine learning*. Available at: <https://pybullet.org> (Accessed: 6 October 2026).
- Dulac-Arnold, G., Levine, N., Mankowitz, D.J., Li, J., Paduraru, C., Goyal, S. and Hester, T. (2021) 'Challenges of real-world reinforcement learning: definitions, benchmarks and analysis', *Machine Learning*, 110(9), pp. 2419–2468.
- Engstrom, L., Ilyas, A., Santurkar, S., Tsipras, D., Janoos, F., Rudolph, L. and Madry, A. (2020) 'Implementation matters in deep policy gradients: a case study on PPO and TRPO', *International Conference on Learning Representations (ICLR 2020)*. Available at: <https://openreview.net/forum?id=r1etN1rtPB> (Accessed: 6 October 2026).
- Haarnoja, T., Zhou, A., Abbeel, P. and Levine, S. (2018) 'Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a stochastic actor', *Proceedings of the 35th International Conference on Machine Learning*, PMLR 80, pp. 1861–1870.
- Haddadin, S., Parusel, S., Johansmeier, L., Golz, S., Gabl, S., Walch, F., Sabaghian, M., Jähne, C., Hausperger, L. and Haddadin, S. (2022) 'The Franka Emika robot: a reference platform for robotics research and education', *IEEE Robotics & Automation Magazine*, 29(2), pp. 46–64.
- Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D. et al. (2020) 'Array programming with NumPy', *Nature*, 585(7825), pp. 357–362.
- Henderson, P., Islam, R., Bachman, P., Pineau, J., Precup, D. and Meger, D. (2018) 'Deep reinforcement learning that matters', *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), pp. 3207–3214.
- Hussein, A., Gaber, M.M., Elyan, E. and Jayne, C. (2017) 'Imitation learning: a survey of learning methods', *ACM Computing Surveys*, 50(2), article 21, pp. 1–35.
- Ibarz, J., Tan, J., Finn, C., Kalakrishnan, M., Pastor, P. and Levine, S. (2021) 'How to train your robot with deep reinforcement learning: lessons we have learned', *The International Journal of Robotics Research*, 40(4–5), pp. 698–721.
- Kalashnikov, D., Irpan, A., Pastor, P., Ibarz, J., Herzog, A., Jang, E. et al. (2018) 'Scalable deep reinforcement learning for vision-based robotic manipulation', *Proceedings of the 2nd Conference on Robot Learning*, PMLR 87, pp. 651–673.
- Khatib, O. (1986) 'Real-time obstacle avoidance for manipulators and mobile robots', *The International Journal of Robotics Research*, 5(1), pp. 90–98.
- Kober, J., Bagnell, J.A. and Peters, J. (2013) 'Reinforcement learning in robotics: a survey', *The International Journal of Robotics Research*, 32(11), pp. 1238–1274.

- Levine, S., Finn, C., Darrell, T. and Abbeel, P. (2016) 'End-to-end training of deep visuomotor policies', *Journal of Machine Learning Research*, 17(39), pp. 1–40.
- Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D. and Wierstra, D. (2016) *Continuous control with deep reinforcement learning*. arXiv:1509.02971. Available at: <https://arxiv.org/abs/1509.02971> (Accessed: 6 October 2026).
- Nair, A., McGrew, B., Andrychowicz, M., Zaremba, W. and Abbeel, P. (2018) 'Overcoming exploration in reinforcement learning with demonstrations', *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 6292–6299.
- Ng, A.Y., Harada, D. and Russell, S. (1999) 'Policy invariance under reward transformations: theory and application to reward shaping', *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pp. 278–287.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G. et al. (2019) 'PyTorch: an imperative style, high-performance deep learning library', *Advances in Neural Information Processing Systems*, 32, pp. 8024–8035.
- Peng, X.B., Andrychowicz, M., Zaremba, W. and Abbeel, P. (2018) 'Sim-to-real transfer of robotic control with dynamics randomization', *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3803–3810.
- Pomerleau, D.A. (1989) 'ALVINN: an autonomous land vehicle in a neural network', *Advances in Neural Information Processing Systems*, 1, pp. 305–313.
- Raffin, A., Hill, A., Gleave, A., Kanervisto, A., Ernestus, M. and Dormann, N. (2021) 'Stable-Baselines3: reliable reinforcement learning implementations', *Journal of Machine Learning Research*, 22(268), pp. 1–8.
- Rajeswaran, A., Kumar, V., Gupta, A., Vezzani, G., Schulman, J., Todorov, E. and Levine, S. (2018) 'Learning complex dexterous manipulation with deep reinforcement learning and demonstrations', *Robotics: Science and Systems XIV*. doi:10.15607/RSS.2018.XIV.049.
- Ross, S., Gordon, G. and Bagnell, D. (2011) 'A reduction of imitation learning and structured prediction to no-regret online learning', *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics*, PMLR 15, pp. 627–635.
- Schaal, S. (1999) 'Is imitation learning the route to humanoid robots?', *Trends in Cognitive Sciences*, 3(6), pp. 233–242.
- Schulman, J., Moritz, P., Levine, S., Jordan, M. and Abbeel, P. (2016) *High-dimensional continuous control using generalized advantage estimation*. arXiv:1506.02438. Available at: <https://arxiv.org/abs/1506.02438> (Accessed: 6 October 2026).
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017) *Proximal policy optimization algorithms*. arXiv:1707.06347. Available at: <https://arxiv.org/abs/1707.06347> (Accessed: 6 October 2026).
- Sutton, R.S. and Barto, A.G. (2018) *Reinforcement learning: an introduction*. 2nd edn. Cambridge, MA: MIT Press.
- Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W. and Abbeel, P. (2017) 'Domain randomization for transferring deep neural networks from simulation to the real world', *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30.
- Towers, M., Kwiatkowski, A., Terry, J., Balis, J.U., De Cola, G., Deleu, T. et al. (2024) *Gymnasium: a standard interface for reinforcement learning environments*. arXiv:2407.17032. Available at: <https://arxiv.org/abs/2407.17032> (Accessed: 6 October 2026).
- Wilson, E.B. (1927) 'Probable inference, the law of succession, and statistical inference', *Journal of the American Statistical Association*, 22(158), pp. 209–212.

- Zhao, W., Queralta, J.P. and Westerlund, T. (2020) 'Sim-to-real transfer in deep reinforcement learning for robotics: a survey', *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, pp. 737–744.
- Zhu, Y., Wang, Z., Merel, J., Rusu, A., Erez, T., Cabi, S., Tunyasuvunakool, S., Kramár, J., Hadsell, R., de Freitas, N. and Heess, N. (2018) 'Reinforcement and imitation learning for diverse visuomotor skills', *Robotics: Science and Systems XIV*. arXiv:1802.09564. Available at: <https://arxiv.org/abs/1802.09564> (Accessed: 6 October 2026).

Appendix A. Reproduction, evidence and supporting data

This appendix lists the commands needed to rerun the main experiments, the scripts of the early development phase, the automated tests, the per-case stress results and the full checkpoint hashes. All paths are relative to the project root. Commands are for Windows PowerShell with Python 3.11.

A.1 Setup and main commands

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install -r requirements.txt
python scripts/check_dependencies.py
```

Table 24. Main command-line entry points (python -m robotic_arm ...).

Command	Purpose and example
run.py	Launcher: starts the recommended shared model in the persistent three-object scene; --headless for a terminal-only run
train	PPO KUKA reaching: train --timesteps 500000 --seed 42 --run-dir outputs/my_reaching_training
evaluate	Evaluate a reaching controller (--controller ppo ik random): evaluate --controller ppo --model models/reaching_policy.zip --episodes 20
record	Record a KUKA reaching episode as a GIF
compare-rewards	Equal-budget legacy-versus-shaped reward comparison (paired continuation seeds)
analyze-failures	Failure diagnostics from the saved reaching report
pick-place, benchmark-pick-place	Scripted physical-gripper baseline and its mass × friction benchmark
train-pick-place	PPO for Panda reach, lift or place stages, or resume from a checkpoint (--resume, --learning-rate, --target-kl)
learn-pick-place	Demonstrations, behaviour cloning, DAGger and optional PPO fine-tuning
refine-obstacle-policy	Focused imitation refinement (--distance-tasks, --random-layouts, --varied-scenes, --align-yaw)
learn-persistent-pick-place	Learned transitions in the fixed three-object scene
train-shared-policy	Balanced behaviour cloning over saved datasets, optional yaw refinement
evaluate-pick-place	Evaluate a Panda checkpoint, e.g. --three-tasks, --random-layouts, --size, --object-yaw, --obstacle-height, --mass, --friction, --domain-randomization, --scene-randomization
benchmark-learned-pick-place	Stress suite: --extended --varied-layouts --episodes-per-case 2 --seed 56000
persistent-pick-place	Three physical objects in one scene without resets
record-pick-place	Record an actual-motion GIF of a policy

To reproduce the final shared refinement from the preserved intermediate checkpoint:

```
python -m robotic_arm train-shared-policy --model
results/shared_simulation_policy/yaw_training/candidate.zip --yaw-refinement --output
outputs/my_shared_refinement
```

To run a new bounded PPO experiment from the shared checkpoint (the experiment of Section 3.9):

```
python -m robotic_arm train-pick-place --stage place --obstacle-aware --random-layouts --
resume models/panda_simulation_policy.zip --learning-rate .00001 --target-kl .001 --timesteps
10240 --output outputs/my_panda_ppo_trial
```

A.2 Development scripts of the early KUKA phase

Table 25. Scripts in the early (v2.8 “watch-precision”) archive.

File	Role
check_install.py	Verifies installed package versions
load_arm_test.py, simulation_test.py	Loads the KUKA arm and lists its joints; drops a cube to test physics
test_action.py	Applies a constant action and confirms that joint 0 moves by about 1 rad over 50 steps
test_env_skeleton.py, test_observation.py	Checks the Gymnasium skeleton and prints the observation vector
test_reward.py, test_success.py	Confirms the reward rises as distance falls; teleports the arm to an IK solution to confirm the success flag
test_random_targets.py, test_full_episodes.py	Random-target and random-action episode checks
environment/reach_env.py	Reaching environment (<i>basic</i> / <i>revec</i> observation)
environment/reach_avoid_env.py, reach_avoid2_env.py	One- and two-sphere obstacle environments
training/train.py	PPO or SAC training with deterministic evaluation callback (--algo, --obs, --seed, --steps; evaluation every 5,000 steps on 30 episodes)
training/train_avoid.py, train_avoid2.py, train_avoid_algo.py	Obstacle-avoidance training variants (one sphere; two spheres; PPO or SAC)
test_trained_policy.py, test_robustness.py, test_avoid_policy.py, test_avoid2_policy.py	Policy evaluation scripts (the robustness script adds observation noise; no saved noise results exist)
watch_policy.py, watch_avoid.py, watch_avoid2.py, watch_avoid_precision.py	GUI viewers for trained policies
compare_ablation.py, plot_training.py, plot_avoid_ablation.py	Ablation and training-curve plots
experiments/pickplace_attempt/	First Panda attempt: pick_place_env.py, train_pickplace.py, test_manual_pickplace.py (IK-scripted success), test_pickplace_policy.py, README documenting both failed PPO runs

A.3 Automated test inventory (42 tests)

Table 26. Unit and integration tests (Python unittest; 42 in total).

Test module	Tests	What is checked
test_reaching_env.py	7	Gym contract; seed and client isolation; action validation; timeout and reset guard; success detection; motor targets within joint limits; render and close
test_pick_place_env.py	19	Space contract and physical object; invalid action and stage; history without contact does not pay or succeed; near-cube reset is not an assisted grasp; repeated contact earns no grasp bonus; ground-sliding penalty and termination; teacher lifts and places without attachments; scene configuration and seeded randomisation; normalised policy transform survives checkpoint; obstacle schema and collision failure; obstacle teacher places large object; extended normalised checkpoint and Gym contract; large-object release opens before withdrawing; quarter-turn teacher regression on six failed scenes; release features preserve earlier predictions; varied-scene geometry and reproducibility; yaw alignment preserves defaults and wrist limits; cross-workspace positions separated and seeded
test_pick_place_controller.py	3	Physical grasp, lift and release; slippery light cube; invalid scene parameters
test_policy_evaluation.py	7	Reproducible IK report; bad episode count; missing checkpoint; window closed before step and between episodes; unrelated physics error not hidden; disconnect during physics call
test_persistent_pick_place.py	4	Task switch keeps world and arm positions; existing reports preserved; camera renders RGB without changing physics; shared policy completes the persistent scene
test_failure_analysis.py	1	Failure report and replay seed
test_model_metadata.py	1	Selected checkpoint SHA-256 hashes match their metadata

Run with `python -m unittest discover -s tests -v`. Integration check: `python scripts/verify_training.py`.

A.4 Per-case stress-suite results across checkpoints

Successes out of two paired episodes in each of the 17 cases, read directly from the saved summary files, and the mean final placement error of the final shared checkpoint. “Yaw dev.” and “Yaw hold.” are the same checkpoint on development seeds (33,000–33,001) and hold-out seeds (34,000–34,001).

Table 27. Successes per case (of 2) for six checkpoints and the final shared checkpoint's mean placement error.

Case	Obst. v2	Varied	Yaw dev.	Yaw hold.	Interm.	+PPO	Final	Final error (cm)
reference	2/2	2/2	1/2	2/2	2/2	2/2	2/2	0.53
light	2/2	2/2	0/2	2/2	2/2	2/2	2/2	0.57
heavy	2/2	2/2	0/2	2/2	2/2	2/2	2/2	1.05
slippery	2/2	2/2	1/2	2/2	2/2	2/2	2/2	0.83
small	2/2	2/2	0/2	2/2	2/2	2/2	2/2	1.43
large	2/2	2/2	1/2	2/2	2/2	2/2	2/2	1.15
cylinder	2/2	2/2	0/2	2/2	2/2	2/2	2/2	1.16
low obstacle	2/2	2/2	0/2	2/2	2/2	2/2	2/2	2.07
blocking obstacle	2/2	2/2	0/2	2/2	2/2	2/2	2/2	0.77
heavy slippery	2/2	2/2	1/2	2/2	2/2	2/2	2/2	0.60
large blocking obstacle	2/2	2/2	0/2	2/2	2/2	2/2	2/2	0.98
cylinder blocking obstacle	0/2	2/2	0/2	2/2	2/2	2/2	2/2	1.97
wide left	0/2	2/2	0/2	2/2	2/2	2/2	2/2	1.16
wide right	0/2	2/2	2/2	2/2	2/2	2/2	2/2	1.26
rotated small box	2/2	2/2	0/2	1/2	1/2	0/2	2/2	1.55
rotated large box	0/2	0/2	2/2	0/2	2/2	2/2	2/2	0.77
shifted obstacle	2/2	2/2	0/2	2/2	2/2	2/2	2/2	1.30
Total	26/34	32/34	8/34	31/34	33/34	32/34	34/34	

A.5 Checkpoint hashes (SHA-256)

Table 28. Full SHA-256 hashes of the evaluated checkpoints, taken from the saved summaries and model metadata. The hashes of the twelve archived checkpoints were re-computed from the model files and match.

Checkpoint	SHA-256
reference_policy.zip	0081c3ac4a59712b10f88dabc7bc55cf24317fccdd3794254a6d72be876147bf
reaching_policy.zip	ab37880c24be3131642125525647036e918dbc1160fc8d0a393ab25a1b964404
panda_pick_place_policy.zip	c7881804ad4a6643f25ba0e00e51af0f5525cb2525f1ec6d60114ef6e54c2f38
panda_lift_policy.zip	f11724fe40c844f4dc5a4364610ad9c2dc807b6d925002b3b3b37a631c0fad70
lift checkpoint (near-cube evaluation)	d47f68d313c221dd9f7cc136b9c50e5c3629369e25df7c190de86aeb786e4ba
panda_randomized_policy.zip	a16a94daf8881c83cfcff66d992300ad75653ff1723e34a7253661ea705f7a9a8
panda_obstacle_policy.zip	a0955be9aaa1cbee7fde35392fb95870b426b5d25d52762a8d9b84fe99818946
panda_obstacle_policy_v2.zip	9315bde0d67d89edb41e2e7324ee686fe0d9bf2d060ccd71bba123646610835c
panda_varied_scene_policy.zip	05a84888d8814dc dab188bc8c0fb3c12fe63662a9deeb2b246b543c050c046e0
panda_diagonal_policy.zip	d34ac53f7dbd9a681545bb4a445f3b88137e3be8b1490ae3bf40b53483b2306d
panda_distance_policy.zip	e1345410bc9fb489b113a9cf6df6a87be8d441a44e9979b93f01f18e448c985a
panda_persistent_policy.zip	0231d8345bd1760afb31ecff9ec0d1a496ff2929a853b5bb9715cec0fac50eed
yaw-aligned candidate (dev./hold-out)	0033b1721eb0408ab4373cbb248120157e00787d6b340d1600c0e86a99e7d6b8
intermediate imitation (before PPO)	8d40cec520ed6a433d0ef737187a84488a9e07a7e10bf4008917ce9ef1a89809
after 10,240 PPO steps	d6f4b646b3d822d03768c55a54d7e7f21e9c10f9672345fbc7b99bffc5d54370
panda_simulation_policy.zip (final shared)	aba0830b49e833ada8687df0e2ae0ad63baa7ad4e9aa73fe48cc2f6e086eb511

A.6 Repository structure

```

robotic-arm-rl/
  robotic_arm/
    environments/ # Physics, observations, scene layouts (reaching, pick-place,
                  # obstacle-aware, oriented, yaw-aligned, persistent)
    controllers/  # Scripted physical-gripper baseline
    training/     # PPO, demonstration learning, refinement, shared policy
    evaluation/   # Learned policies, persistent sequences, recordings
    experiments/  # Benchmarks, reward comparison, failure analysis
    utils/        # Output protection (no overwrite)
    tests/        # 42 automated tests
    scripts/      # check_dependencies, verify_training, plot_panda_results
    models/       # Checkpoints and *_selection.json metadata
    results/      # Immutable experiment evidence
    docs/media/   # Actual simulation recordings (GIF)
    outputs/      # Generated local runs (excluded from Git)
  requirements.txt README.md run.py

```

A.7 Glossary

- **BC:** behaviour cloning, supervised learning of actions from demonstrations.
- **Dagger:** dataset aggregation, iterative imitation in which the expert labels states visited by the learner.
- **Domain randomisation:** varying simulator parameters (mass, friction, size, shape) during training or testing.

- **End effector:** the tool-end of the arm; here the last KUKA link or Panda link 11.
- **GAE:** generalised advantage estimation.
- **IK:** inverse kinematics, computing joint angles for a desired end-effector pose.
- **PPO / SAC:** proximal policy optimization / soft actor-critic.
- **Relvec:** observation that adds the target-minus-end-effector vector.
- **Stress suite:** a finite set of test conditions evaluated on paired seeds.
- **Wilson interval:** a binomial confidence interval that behaves well for small samples and proportions near 0 or 1.
- **Yaw:** rotation about the vertical axis.